

# Real-Time FPGA Electromagnetic Field Visualiser

Team: On the Edge

Marzouk Hajaij

Taha Munir  
Mingze Chen

Cyril Abdel Nour  
Yi Dong

Run Zhang

June 2026

**GitHub:** <https://github.com/TahaMunir2/ElectroMagnetic-FPGA-3D-Renderer>

**Video:** <https://youtu.be/cKno8BfnNS0>

**Word count:** 9,927

# Contents

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The Problem: Visualising the Invisible</b>                              | <b>4</b>  |
| 1.1      | Project specifications and requirements . . . . .                          | 5         |
| <b>2</b> | <b>Overall Architecture and Project Management</b>                         | <b>6</b>  |
| 2.1      | System architecture . . . . .                                              | 6         |
| 2.2      | Project Management . . . . .                                               | 7         |
| 2.2.1    | Team structure . . . . .                                                   | 7         |
| 2.2.2    | Working practices . . . . .                                                | 7         |
| 2.2.3    | Deliverable structure . . . . .                                            | 7         |
| 2.2.4    | Timeline . . . . .                                                         | 8         |
| <b>3</b> | <b>The FDTD Solver and Physics</b>                                         | <b>9</b>  |
| 3.1      | Maxwell's Equations . . . . .                                              | 9         |
| 3.1.1    | Meaning of each symbol . . . . .                                           | 9         |
| 3.1.2    | Why we set $\mathbf{J} = \mathbf{0}$ . . . . .                             | 9         |
| 3.1.3    | Dynamic case: the wave equation ( $\partial/\partial t \neq 0$ ) . . . . . | 10        |
| 3.1.4    | Which equations are actively solved in the FDTD engine . . . . .           | 10        |
| 3.2      | Field Component Reduction: From Six to Three . . . . .                     | 10        |
| 3.2.1    | Step 1: The full 3D coupling . . . . .                                     | 10        |
| 3.2.2    | Step 2: Applying translational symmetry in $z$ . . . . .                   | 11        |
| 3.2.3    | Step 3: The two decoupled field systems . . . . .                          | 12        |
| 3.3      | Why Two Dimensions . . . . .                                               | 13        |
| 3.4      | Discretisation . . . . .                                                   | 13        |
| 3.5      | Choosing the Algorithm . . . . .                                           | 15        |
| 3.6      | The Coefficients and Their Limits . . . . .                                | 16        |
| 3.7      | Number Format . . . . .                                                    | 16        |
| 3.8      | Pipelining the Update . . . . .                                            | 17        |
| 3.9      | Boundaries and the Absorbing Layer . . . . .                               | 18        |
| 3.10     | From Three Passes to Two . . . . .                                         | 19        |
| 3.11     | Lane Parallelism and the Halo . . . . .                                    | 20        |
| 3.12     | Performance Comparison . . . . .                                           | 21        |
| 3.13     | Verification . . . . .                                                     | 21        |
| <b>4</b> | <b>The Renderer</b>                                                        | <b>23</b> |
| 4.1      | Foundations and the baseline design . . . . .                              | 23        |
| 4.1.1    | The rendering problem . . . . .                                            | 23        |
| 4.1.2    | The fixed pixel pipeline . . . . .                                         | 23        |
| 4.1.3    | Number representation . . . . .                                            | 24        |
| 4.1.4    | Choosing the march depth . . . . .                                         | 24        |
| 4.1.5    | The baseline architecture . . . . .                                        | 25        |
| 4.2      | Shading interpolation . . . . .                                            | 27        |

|          |                                                                                    |           |
|----------|------------------------------------------------------------------------------------|-----------|
| 4.2.1    | Reusing the reads the normal already makes . . . . .                               | 27        |
| 4.2.2    | The interpolation mathematics . . . . .                                            | 27        |
| 4.2.3    | Cost . . . . .                                                                     | 28        |
| 4.3      | Folding and the FIFO scanout . . . . .                                             | 29        |
| 4.3.1    | Clocking and the two domains . . . . .                                             | 29        |
| 4.3.2    | Matching throughput and latency to the display . . . . .                           | 29        |
| 4.3.3    | Reliability . . . . .                                                              | 30        |
| 4.3.4    | The limit of the FIFO scanout . . . . .                                            | 30        |
| 4.4      | Silhouette interpolation and the folded board design . . . . .                     | 30        |
| 4.4.1    | Bilinear in the marcher, and the cost explosion . . . . .                          | 30        |
| 4.4.2    | Folding the reads . . . . .                                                        | 31        |
| 4.4.3    | Folding the multipliers: from 70% to 26% DSP . . . . .                             | 31        |
| 4.4.4    | Scaling to $128 \times 128$ is still not possible without parallel lanes . . . . . | 32        |
| 4.5      | The frame buffer and parallel lanes . . . . .                                      | 32        |
| 4.5.1    | Decoupling with a frame buffer . . . . .                                           | 33        |
| 4.5.2    | Recovering speed with parallel lanes . . . . .                                     | 33        |
| 4.5.3    | Why this uses fewer resources . . . . .                                            | 34        |
| 4.6      | Final comparison: five candidate designs . . . . .                                 | 34        |
| 4.6.1    | Fitting the DSP budget . . . . .                                                   | 35        |
| 4.6.2    | The final design . . . . .                                                         | 35        |
| 4.6.3    | Renderer performance . . . . .                                                     | 35        |
| <b>5</b> | <b>Interface and System Integration</b>                                            | <b>37</b> |
| 5.1      | System Overview . . . . .                                                          | 37        |
| 5.2      | Building the Design from a Script . . . . .                                        | 37        |
| 5.3      | Clocking . . . . .                                                                 | 38        |
| 5.4      | Generating the Source . . . . .                                                    | 39        |
| 5.5      | The Moving Source and the Doppler Effect . . . . .                                 | 39        |
| 5.6      | Decoupling the Solver and the Display . . . . .                                    | 40        |
| 5.7      | The Bridge to the Renderer . . . . .                                               | 41        |
| 5.8      | Driving It from the Processor . . . . .                                            | 41        |
| 5.9      | Multi-Lane Parallelism . . . . .                                                   | 41        |
| 5.10     | The Two Demonstrations . . . . .                                                   | 42        |
| <b>6</b> | <b>User Interface, Hardware and Demonstrations</b>                                 | <b>45</b> |
| 6.1      | What the demonstrator does, and the physics behind it . . . . .                    | 45        |
| 6.2      | The user interface . . . . .                                                       | 45        |
| 6.3      | Power supply and analog conditioning . . . . .                                     | 45        |
| 6.4      | The microcontroller and the control link . . . . .                                 | 48        |
| 6.5      | The touch panel: sensing and calibration . . . . .                                 | 48        |
| 6.6      | The web interface . . . . .                                                        | 49        |
| 6.6.1    | Requirements . . . . .                                                             | 49        |
| 6.6.2    | Architecture: two data planes . . . . .                                            | 50        |
| 6.6.3    | One simulation, two control surfaces . . . . .                                     | 51        |
| 6.6.4    | Designed as a teaching instrument . . . . .                                        | 52        |
| 6.7      | Enclosure and mechanical design (box modelling) . . . . .                          | 54        |
| 6.8      | Interactive Wave Simulation . . . . .                                              | 55        |
| 6.8.1    | Panel 1: Electric field $ \mathbf{E} (x, y, t)$ . . . . .                          | 55        |
| 6.8.2    | Panel 2: Magnetic field $B_z(x, y, t)$ . . . . .                                   | 55        |
| 6.8.3    | Panel 3: Poynting vector $ \mathbf{S} (x, y, t)$ . . . . .                         | 58        |
| 6.8.4    | Relationship between the three panels . . . . .                                    | 58        |
| 6.8.5    | More demonstrations . . . . .                                                      | 58        |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>7</b> | <b>Evaluation and Conclusion</b>        | <b>59</b> |
| 7.1      | Integrated system performance . . . . . | 59        |
| 7.2      | Initial vs. final design . . . . .      | 60        |
| 7.3      | Limitations and future work . . . . .   | 60        |



# Chapter 1

## The Problem: Visualising the Invisible

**Abstract.** This report details the physics underlying a real-time, FPGA-accelerated electromagnetic-field visualiser designed as an educational tool for electrical engineering students. The system makes Maxwell's equations physically tangible by solving the time-dependent field equations numerically and rendering the resulting fields as interactive 3D surfaces. Although Maxwell's equations can be solved analytically in simple cases, the fields themselves are invisible and difficult to interpret intuitively. The visualiser addresses this problem by allowing students to place wave sources, add reflecting boundaries, vary material properties, and observe propagation, interference, diffraction, and energy flow in real time.

## 1.1 Project specifications and requirements

| Requirement                                                                                              | How the project addresses it                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Visualisation</i>                                                                                     |                                                                                                                                                                                                                                                        |
| Display a visualisation of a mathematical function computed in real time.                                | The FDTD solution of Maxwell’s equations is rendered to HDMI at $640 \times 480$ , 60 fps, updating live (Ch. 3–5).                                                                                                                                    |
| The function shall be computationally intensive — non-trivial at the required resolution and frame rate. | A $128 \times 128$ field is time-stepped every frame and ray-marched per pixel; the per-pixel renderer alone is the binding throughput constraint (Ch. 4).                                                                                             |
| The computation should be embarrassingly parallel (pixels independent).                                  | The renderer ray-marches every pixel independently with no inter-pixel data dependence, and the FDTD field has only local nearest-neighbour coupling and is parallelised by spatial decomposition (Ch. 3, Ch. 4).                                      |
| <i>Accelerator</i>                                                                                       |                                                                                                                                                                                                                                                        |
| Generated on the PYNQ-Z1, with an accelerator in FPGA soft logic computing the inner loops.              | The FDTD update and the ray-march inner loops both run in the programmable logic; the processor only configures and reads back (Ch. 3–5).                                                                                                              |
| The accelerator shall be described in Verilog or SystemVerilog.                                          | All datapath RTL (solver, magnitude, bridge, renderer) is SystemVerilog/Verilog, built from a scripted, version-controlled flow (Ch. 5).                                                                                                               |
| The accelerator shall provide a CPU interface for parameter adjustment.                                  | An AXI-GPIO register map exposes amplitude, frequency, source position, display mode, motion and camera to the processor, driven live from a UI server (Ch. 5).                                                                                        |
| Number formats and word lengths chosen to trade accuracy against throughput.                             | Signed Q3.13 fixed-point in solver and various number 8 and 16 bit formats used in renderer, selected from a documented comparison of formats (Q1.15–FP32) against rounding error and resource cost (Ch. 3).                                           |
| Throughput shall exceed a CPU-only C/C++/Cython implementation.                                          | Benchmarked against an ARM (NEON) software baseline, the accelerated solver is substantially faster (Ch. 3, Ch. 4, Ch. 7).                                                                                                                             |
| <i>User interface</i>                                                                                    |                                                                                                                                                                                                                                                        |
| Provide a user interface that enhances educational value.                                                | The user can place and move sources, switch field/energy views, vary speed and material, and watch propagation, interference, the Doppler effect and energy flow in real time as well as adjust the camera position with 5 degrees of freedom (Ch. 6). |
| The UI may use separate hardware.                                                                        | A standalone ESP32 with a resistive touch panel, linked to the board over serial (Ch. 6).                                                                                                                                                              |
| The UI shall allow intuitive or interesting parameter adjustment.                                        | The source is placed by pressing a touch panel, potentiometers set amplitude, material and camera (Ch. 6).                                                                                                                                             |
| The UI should present information as an overlay or a separate medium.                                    | Parameters and status are shown on the host notebook (separate medium), the physical panel is labelled, and a colour-vision-safe palette communicates field magnitude (Ch. 6).                                                                         |

Table 1.1: Project requirements and where each is addressed. Quantitative evidence that they were met is given in the Evaluation chapter.

## Chapter 2

# Overall Architecture and Project Management

### 2.1 System architecture

The complete system is a single streaming pipeline on the PYNQ-Z1, shown in Figure 2.1. The processor configures the programmable logic, the source generator feeds the FDTD solver, the field is double-buffered and bridged into a heightmap, and the ray-march renderer draws it out over HDMI alongside a web UI.

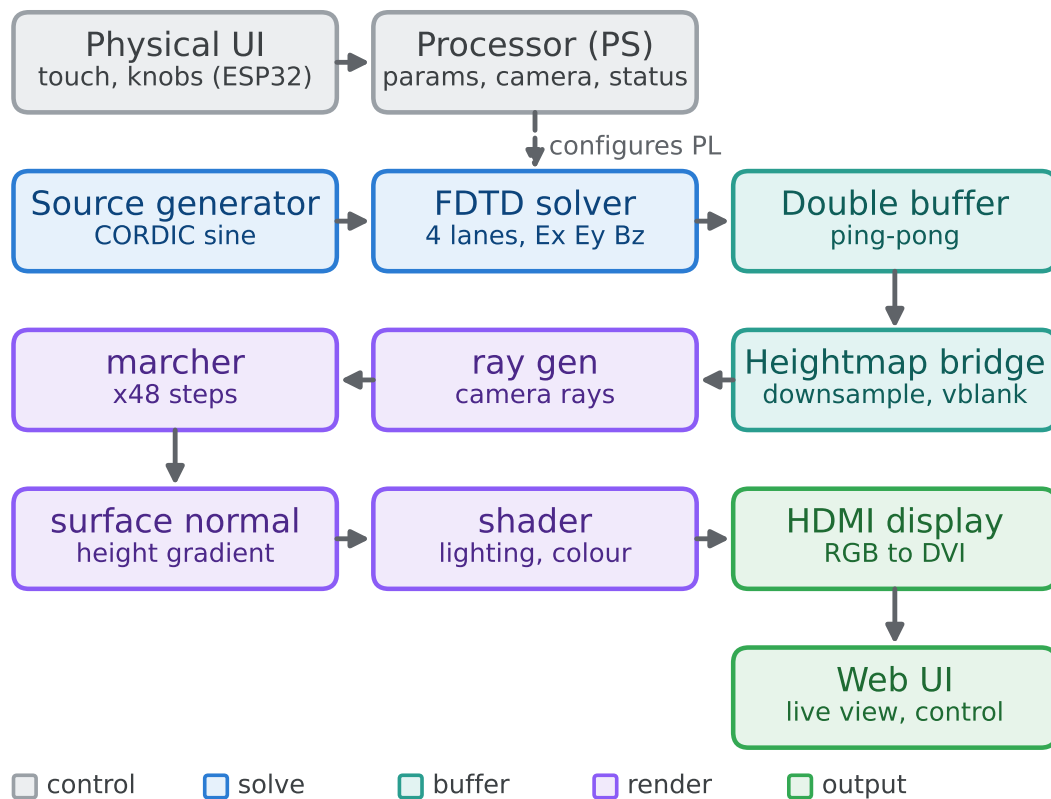


Figure 2.1: End-to-end system architecture, coloured by function: control, solve, buffer, render and output.

## 2.2 Project Management

### 2.2.1 Team structure

The project was divided into three sub-teams, each responsible for a distinct subsystem. Table 2.1 lists each member’s contributions.

| Name             | Team      | Contributions                                                                                      |
|------------------|-----------|----------------------------------------------------------------------------------------------------|
| Taha Munir       | FDTD      | FDTD solver implementation, PML physics, parallelisation of FDTD engine                            |
| Yi Dong          | FDTD      | Vivado/Tcl block design, AXI control plane, ping-pong buffer, CORDIC source, field post-processing |
| Run Zhang        | Analog    | Hardware probing, physical interface, wiring design and CAD assistance                             |
| Marzouk Hajaij   | Analog    | Analog circuit design, physics implementation, 3D visualisation                                    |
| Cyril Abdel Nour | Rendering | Web interface and FPGA link, FPGA ray-marching 3D renderer design and optimisation                 |
| Mingze Chen      | Rendering | 3D renderer design and optimisation, PYNQ deployment and testing, 3D-printed enclosure             |

Table 2.1: Division of work across the three sub-teams.

### 2.2.2 Working practices

**Work Split.** The group was split into three sub-teams that worked individually on their parts, while remaining in contact with the other sub-teams to make sure everything moved together. Questions between sub-teams were answered right away, ensuring that everything was tied together. This led to the sub-teams, especially the rendering and FDTD teams, working in the same space many times.

**Meetings and Collaborations.** Group meetings were held every couple of days to see where every sub-team had reached and what they were planning to get done next. This allowed updates on the project’s path.

**Github.** The codebase was managed on Git with one branch per sub-team. The main branch was kept always-buildable. Merges were conducted via peer-reviewed pull requests, so parallel development across three sub-teams never broke the working skeleton.

### 2.2.3 Deliverable structure

Each sub-team worked toward four internal deliverables, providing intermediate checkpoints against the project timeline as shown in Figure 2.2.

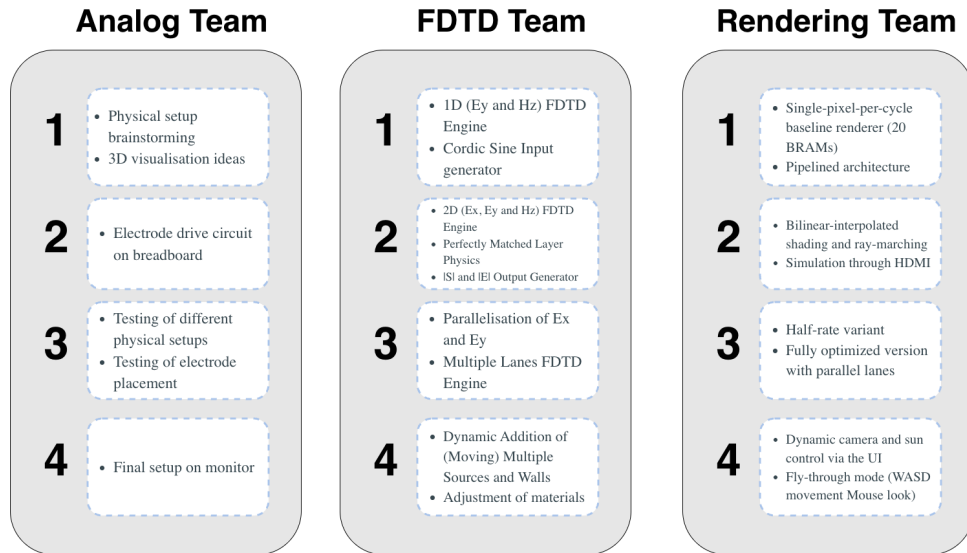


Figure 2.2: Deliverable / MVP roadmap across the three sub-teams.

#### 2.2.4 Timeline

The timeline to complete these deliverables is shown in Figure 2.3.

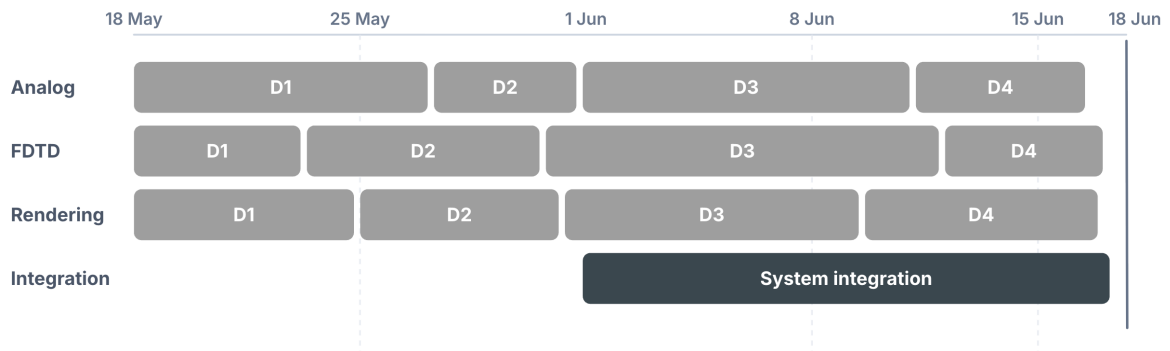


Figure 2.3: Project timeline across the three sub-teams.

## Chapter 3

# The FDTD Solver and Physics

### 3.1 Maxwell's Equations

Our entire system relies on actively solving Maxwell's equations. In differential form:

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0} \quad (\text{Gauss – electric})$$

$$\nabla \cdot \mathbf{B} = 0 \quad (\text{Gauss – magnetic})$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \quad (\text{Faraday})$$

$$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t} \quad (\text{Ampère–Maxwell})$$

#### 3.1.1 Meaning of each symbol

The symbols in the equations above are defined as follows.

$\mathbf{E}$  ( $\text{V m}^{-1}$ ) is the electric field vector.

$\mathbf{B}$  (T) is the magnetic flux density.

$\rho$  ( $\text{C m}^{-3}$ ) is the free electric charge density.  $\mathbf{J}$  ( $\text{A m}^{-2}$ ) is the current density. It is the amount of charge that passes through an elementary plate in the unit of time and in the direction perpendicular to the plate.

$\varepsilon_0 = 8.85 \times 10^{-12} \text{ F m}^{-1}$  is the permittivity of free space.

$\mu_0 = 4\pi \times 10^{-7} \text{ H m}^{-1}$  is the permeability of free space.

$$c = \frac{1}{\sqrt{\mu_0 \varepsilon_0}} \approx 3 \times 10^8 \text{ m s}^{-1} \quad (3.1)$$

The divergence  $\nabla \cdot$  shows if the field is converging or diverging and by how much.

The curl  $\nabla \times$  has a magnitude which shows the amount of rotation of the field at a particular point.

#### 3.1.2 Why we set $\mathbf{J} = 0$

Setting  $\mathbf{J} = 0$  means we are simulating a region with no free currents flowing through it. This makes sense because the simulation domain is free space or a dielectric medium, so there are no wires, conductors, and charged particles moving through the field.

With  $\mathbf{J} = 0$  everywhere, the Ampère–Maxwell equation simplifies to:

$$\nabla \times \mathbf{B} = \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t} \quad (3.2)$$

Similarly, in a charge-free medium ( $\rho = 0$ ), Gauss's law for electricity becomes simply  $\nabla \cdot \mathbf{E} = 0$ .

### 3.1.3 Dynamic case: the wave equation ( $\partial/\partial t \neq 0$ )

When fields vary in time, Faraday's equation and the simplified Ampère–Maxwell equation (3.2) are coupled. Taking the curl of the Faraday equation:

$$\nabla \times (\nabla \times \mathbf{E}) = -\frac{\partial}{\partial t} \nabla \times \mathbf{B} = -\mu_0 \varepsilon_0 \frac{\partial^2 \mathbf{E}}{\partial t^2} \quad (3.3)$$

Applying the vector identity  $\nabla \times (\nabla \times \mathbf{E}) = \nabla(\nabla \cdot \mathbf{E}) - \nabla^2 \mathbf{E}$  and using  $\nabla \cdot \mathbf{E} = 0$  in a charge-free region:

$$\boxed{\nabla^2 \mathbf{E} = \mu_0 \varepsilon_0 \frac{\partial^2 \mathbf{E}}{\partial t^2}} \quad (3.4)$$

This is the electromagnetic wave equation. The wave speed in free space is  $c = 1/\sqrt{\mu_0 \varepsilon_0}$  as given in (3.1). In a medium with relative permittivity  $\varepsilon_r$  and relative permeability  $\mu_r$ :

$$v = \frac{c}{\sqrt{\varepsilon_r \mu_r}} \quad (3.5)$$

### 3.1.4 Which equations are actively solved in the FDTD engine

The four Maxwell equations do not all require explicit solution in the FDTD loop. The two divergence equations, Gauss's law (Gauss – electric) and the magnetic Gauss law (Gauss – magnetic), are automatically satisfied for all time as a consequence of the curl equations, as the fields start at 0 everywhere.

The FDTD engine therefore explicitly solves only two equations:

- The Faraday equation, which updates  $B_z$  from the curl of  $\mathbf{E}$ .
- The simplified Ampère–Maxwell equation (3.2), which updates  $E_x$  and  $E_y$  from the gradient of  $B_z$ .

## 3.2 Field Component Reduction: From Six to Three

### 3.2.1 Step 1: The full 3D coupling

In full three-dimensional space, Maxwell's curl equations couple all six field components together. Writing the Faraday equation  $\nabla \times \mathbf{E} = -\partial \mathbf{B}/\partial t$  and the Ampère–Maxwell equation  $\nabla \times \mathbf{B} = \mu_0 \varepsilon_0 \partial \mathbf{E}/\partial t$  in component form gives six equations.

**From Faraday's law:**

$$\frac{\partial B_x}{\partial t} = \frac{\partial E_y}{\partial z} - \frac{\partial E_z}{\partial y} \quad (3.6)$$

$$\frac{\partial B_y}{\partial t} = \frac{\partial E_z}{\partial x} - \frac{\partial E_x}{\partial z} \quad (3.7)$$

$$\frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \quad (3.8)$$

**From Ampère–Maxwell:**

$$\frac{\partial E_x}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \frac{\partial B_z}{\partial y} - \frac{\partial B_y}{\partial z} \right) \quad (3.9)$$

$$\frac{\partial E_y}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \frac{\partial B_x}{\partial z} - \frac{\partial B_z}{\partial x} \right) \quad (3.10)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \quad (3.11)$$

To update any single component requires two others.

### 3.2.2 Step 2: Applying translational symmetry in $z$

Our simulation domain is two-dimensional and the model is assumed to be uniform in the out-of-plane  $z$  direction, so every  $z$ -partial derivative is identically zero:

$$\frac{\partial}{\partial z} = 0 \quad (3.12)$$

**Faraday equations after  $\partial/\partial z = 0$ :**

$$\frac{\partial B_x}{\partial t} = \cancel{\frac{\partial E_y}{\partial z}} - \frac{\partial E_z}{\partial y} \longrightarrow \frac{\partial B_x}{\partial t} = -\frac{\partial E_z}{\partial y} \quad (3.13)$$

$$\frac{\partial B_y}{\partial t} = \frac{\partial E_z}{\partial x} - \cancel{\frac{\partial E_x}{\partial z}} \longrightarrow \frac{\partial B_y}{\partial t} = +\frac{\partial E_z}{\partial x} \quad (3.14)$$

$$\frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \longrightarrow \frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \quad (\text{unchanged}) \quad (3.15)$$

**Ampère–Maxwell equations after  $\partial/\partial z = 0$ :**

$$\frac{\partial E_x}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \frac{\partial B_z}{\partial y} - \cancel{\frac{\partial B_y}{\partial z}} \right) \longrightarrow \frac{\partial E_x}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \frac{\partial B_z}{\partial y} \quad (3.16)$$

$$\frac{\partial E_y}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \cancel{\frac{\partial B_x}{\partial z}} - \frac{\partial B_z}{\partial x} \right) \longrightarrow \frac{\partial E_y}{\partial t} = -\frac{1}{\mu_0 \varepsilon_0} \frac{\partial B_z}{\partial x} \quad (3.17)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \longrightarrow \frac{\partial E_z}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left( \frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \quad (\text{unchanged}) \quad (3.18)$$

The system has split into two completely independent sets of three equations. An example of the components of the wave can be seen in the figure below:



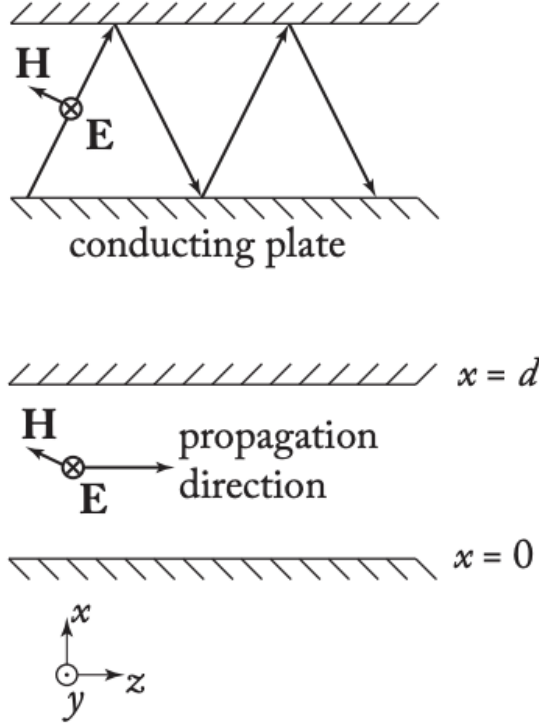


Figure 3.1: TE-polarised propagation between two parallel boundaries. This has the same field-component geometry as the TE system  $(E_x, E_y, B_z)$  used in our 2D FDTD simulation.

### 3.2.3 Step 3: The two decoupled field systems

The six equations separate exactly into two independent systems. This is an exact result, not an approximation.

**TM system**  $(E_z, B_x, B_y)$ :

$$\frac{\partial B_x}{\partial t} = -\frac{1}{\mu_0} \frac{\partial E_z}{\partial y} \quad (3.19)$$

$$\frac{\partial B_y}{\partial t} = +\frac{1}{\mu_0} \frac{\partial E_z}{\partial x} \quad (3.20)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon_0} \left( \frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \quad (3.21)$$

**TE system**  $(E_x, E_y, B_z)$ :

$$\frac{\partial B_z}{\partial t} = -\frac{1}{\mu_0} \left( \frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} \right) \quad (3.22)$$

$$\frac{\partial E_x}{\partial t} = +\frac{1}{\varepsilon_0} \frac{\partial B_z}{\partial y} \quad (3.23)$$

$$\frac{\partial E_y}{\partial t} = -\frac{1}{\varepsilon_0} \frac{\partial B_z}{\partial x} \quad (3.24)$$

$E_z$  points perpendicular to the simulation plane.  $B_x$  and  $B_y$  lie within the plane.

$E_x$  and  $E_y$  lie within the simulation plane.  $B_z$  points perpendicular to the plane.

If the TE system is initialised to zero and only the TM system is excited, the TE components remain zero for all time, and vice versa. We simulate only the TE system, so we ignore the TM components.

The TE system, storing  $E_x$ ,  $E_y$ , and  $B_z = \mu_0 H_z$ , was selected for three reasons:

1. **Poynting vector lies in-plane:** The electromagnetic power flux  $\mathbf{S} = \mathbf{E} \times \mathbf{B}/\mu_0$  for the TE system has components:

$$S_x = \frac{E_y B_z}{\mu_0}, \quad S_y = -\frac{E_x B_z}{\mu_0}, \quad S_z = 0 \quad (3.25)$$

Both  $S_x$  and  $S_y$  lie within the  $xy$  plane and can be visualised directly. For the TM system, the in-plane energy-flow representation is less direct for this visualisation.

2. **Natural representation on a 2D grid:** The electric-field components  $E_x$  and  $E_y$  lie directly in the simulation plane, while  $B_z$  is perpendicular to it. This arrangement matches the Cartesian grid and allows field direction, magnitude, and energy flow to be computed without reconstructing missing in-plane components.
3. **Nicer visualisation:** Two electric field components give both magnitude  $|\mathbf{E}| = \sqrt{E_x^2 + E_y^2}$  and direction  $\theta = \arctan(E_y/E_x)$ . The TM system provides only the scalar  $E_z$ , giving magnitude alone.

All electromagnetism material and information in this section, including equations, derivations, and the TE-polarisation diagram (Figure 3.1) are drawn from the second-year electromagnetism lecture notes.

### 3.3 Why Two Dimensions

The TE equations solved by our FDTD algorithm are:

$$\begin{aligned}\epsilon \frac{\partial E_x}{\partial t} &= \frac{\partial H_z}{\partial y} \\ \epsilon \frac{\partial E_y}{\partial t} &= -\frac{\partial H_z}{\partial x} \\ \mu \frac{\partial H_z}{\partial t} &= \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x}\end{aligned}$$

The reason for choosing 3 fields in 2 dimensions instead of 6 fields in 3 dimensions is because of our strict memory limitation of 4.9 Mbits of BRAM.

|    | Grid     | Fields                         | BRAM36 | Fraction of 140  |
|----|----------|--------------------------------|--------|------------------|
| 2D | 64×64    | $E_x, E_y, B_z$                | 6      | 4%               |
| 2D | 128×128  | $E_x, E_y, B_z$                | 24     | 16%              |
| 3D | 64×64×64 | $E_x, E_y, E_z, H_x, H_y, H_z$ | 768    | 549% (5.5× over) |

Table 3.1: Memory cost of 2D vs 3D FDTD at 16-bit (Q3.13)

These calculations do not account for memory needed for height maps, double buffers, and the significant memory demands of the renderer, nor for the fact that our final grid length is 128. Even with creative use of DDR caching it is not feasible to compute FDTD in 3 dimensions on the PYNQ-Z1. DDR caching was determined not to be useful anyway as the rendering logic needs random access which would defeat the purpose of DDR caching.

### 3.4 Discretisation

Maxwell’s curl equations are PDEs, which are extremely expensive to compute on the PL, especially given the number we need to evaluate. We therefore chose to use the industry-renowned

FDTD method to discretise them. It uses the following second-order-accurate approximation, obtained from the Taylor series expansion of the partial derivative.

$$\frac{\partial H}{\partial x} \approx \frac{H(x + \frac{\Delta x}{2}) - H(x - \frac{\Delta x}{2})}{\Delta x}$$

Derivation of FDTD form for Ex:

$$\varepsilon_0 \frac{\partial E_x}{\partial t} = \frac{\partial H_z}{\partial y} \quad (3.26)$$

Divide by  $\varepsilon_0$ , substitute  $H_z = B_z/\mu_0$ , normalise  $\varepsilon_0\mu_0 = c^{-2} = 1$ :

$$\frac{\partial E_x}{\partial t} = \frac{\partial B_z}{\partial y} \quad (3.27)$$

Taylor expand  $B_z$  about  $[i, j]$  at  $\pm \frac{\Delta y}{2}$  in the row ( $i$ ) direction:

$$B_z\left[i + \frac{1}{2}, j\right] = B_z[i, j] + \frac{\Delta y}{2} \frac{\partial B_z}{\partial y} + \frac{1}{2} \left(\frac{\Delta y}{2}\right)^2 \frac{\partial^2 B_z}{\partial y^2} + O(\Delta y^3) \quad (3.28)$$

$$B_z\left[i - \frac{1}{2}, j\right] = B_z[i, j] - \frac{\Delta y}{2} \frac{\partial B_z}{\partial y} + \frac{1}{2} \left(\frac{\Delta y}{2}\right)^2 \frac{\partial^2 B_z}{\partial y^2} + O(\Delta y^3) \quad (3.29)$$

Subtract:

$$B_z\left[i + \frac{1}{2}, j\right] - B_z\left[i - \frac{1}{2}, j\right] = \Delta y \frac{\partial B_z}{\partial y} + O(\Delta y^3) \quad (3.30)$$

Dividing by  $\Delta y$  and rearranging:

$$\frac{\partial B_z}{\partial y} = \frac{B_z[i, j] - B_z[i - 1, j]}{\Delta y} + O(\Delta y^2) \quad (3.31)$$

Substituting into (2) and applying the same argument in time:

$$\frac{E_x^{n+1}[i, j] - E_x^n[i, j]}{\Delta t} = \frac{B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j]}{\Delta y} + O(\Delta t^2, \Delta y^2) \quad (3.32)$$

Rearrange:

$$E_x^{n+1}[i, j] = E_x^n[i, j] + \frac{\Delta t}{\Delta y} \left( B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j] \right) \quad (3.33)$$

Add PML coefficients  $c_a$  and  $c_b$ :

$$E_x^{n+1}[i, j] = c_a[i, j] \cdot E_x^n[i, j] - c_b \left( B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j] \right) \quad (3.34)$$

FDTD uses a Yee cell lattice to represent the magnetic and electric fields in 2D space with a half-cell staggered approach, meaning the magnetic fields are at half-integer indices while the electric fields are at integer indices. This means that  $E_x$  cells are updated by the change in their top and bottom  $B_z$  components,  $E_y$  cells are updated by the change in their left and right  $B_z$

components, and  $B_z$  cells are updated by the change in the surrounding two  $E_x$  and two  $E_y$  components.

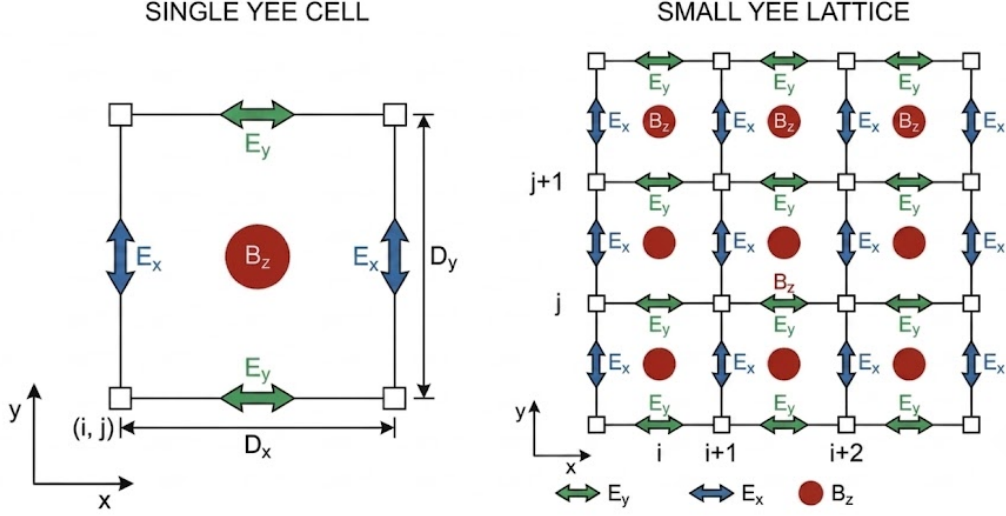


Figure 3.2: Yee cell lattice showing the half-cell staggering of the  $E_x$ ,  $E_y$  and  $B_z$  field components

|              | $E_y[i][j]$ | $E_x[i][j]$ | $B_z[i][j]$ | $B_z[i-1][j]$ | $B_z[i][j-1]$ | $E_y[i][j+1]$ | $E_x[i+1][j]$ |
|--------------|-------------|-------------|-------------|---------------|---------------|---------------|---------------|
| $E_y$ update | ×           |             | ×           |               | ×             |               |               |
| $E_x$ update |             | ×           | ×           | ×             |               |               |               |
| $B_z$ update | ×           | ×           | ×           |               |               | ×             | ×             |

Table 3.2: Read dependencies for each field update in the  $TE_z$  FDTD scheme ( $i$  = row,  $j$  = column).

This method relies on computing the entire electric field and then computing the entire magnetic field, as each field only changes when the other field changes, so they can be computed independently.

If a source is dropped into the central  $E_y$  cell, then in the first magnetic pass its adjacent  $B_z$  fields will update with non-zero values; in the next electric pass the neighbouring  $E_x$  and  $E_y$  cells will update, and so on, propagating the wave outwards from that point.

### 3.5 Choosing the Algorithm

We chose FDTD over alternative algorithms due to its synergy with an FPGA implementation. An FDFD algorithm would require random access, which is inefficient with our BRAM constraints (as we already have a random-access renderer) and FFT methods require all other cells to calculate any single cell which is not the most efficient for an FPGA implementation. FDTD provided a

solution that was sufficiently accurate while being suitable for parallelisation and our BRAM budget.

### 3.6 The Coefficients and Their Limits

The coefficient  $c_a$  determines how much of a cell's own value it keeps each timestep, so in a vacuum  $c_a = 1$  and the field only changes through its neighbours, while  $c_a < 1$  models a lossy medium which is what the PML uses.  $c_b$  sets how strongly the neighbour difference drives the update and gives us the Courant number  $S = |c_b|/2^{13}$ , the fraction of a cell the wave moves per iteration. This is bounded above by the CFL stability condition which requires  $S \leq 1/\sqrt{2}$  in 2D, as any faster causes the leapfrog scheme to amplify instead of propagate and overflow the Q3.13 range. It is bounded below by quantisation, since  $c_b$  is an integer number of Q3.13 steps and the coupling term rounds to zero once it gets too small. We chose  $c_b = -717$  ( $S \approx 0.0875$ ), large enough that the wave propagates cleanly while staying well inside the CFL limit.

### 3.7 Number Format

To prevent segmentation in the BRAM a power of 2 would have been ideal for the number format, so we chose 16 bits as it reduces the memory from the standard 32-bit floating-point format while maintaining sufficient accuracy to demonstrate our results. We chose Q3.13 as it covers the range  $\pm 4$  (sufficiently large for any constructive interference) while maximising fineness of resolution (0.00012). To tackle the problem of constraining a 32 bit multiplication result back into a 16 bit format we took bits [28:13]. Taking these bits alone truncates towards negative infinity, which biases every multiplication by up to one LSB in the same direction. In the interior  $c_a = 1$  leaves nothing to damp this, so the bias accumulates over thousands of iterations into a visible artifact in the visualisation. We fixed this by adding half an LSB before taking the bits, which rounds to nearest and removes the bias, leaving only a negligible truncation error ( $\sim 10^{-4}$ ) and a small error/frame that we deemed a suitable trade off.

| Format | Range   | Error/frame | Note                                       |
|--------|---------|-------------|--------------------------------------------|
| Q1.15  | $\pm 1$ | 0.04%       | range too small, clips on interference     |
| Q2.14  | $\pm 2$ | 0.09%       | range still too small in edge cases        |
| Q3.13  | $\pm 4$ | 0.17%       | our choice, enough range and low error     |
| Q4.12  | $\pm 8$ | 0.34%       | wasted range and double the error          |
| Q3.21  | $\pm 4$ | <0.01%      | lower error but 24 bits fragments the BRAM |
| FP32   | huge    | $\sim 0\%$  | negligible error but double the memory     |

Table 3.3: The number formats we considered. Error/frame is the rounding error accumulated over one 812 iteration frame.

### 3.8 Pipelining the Update

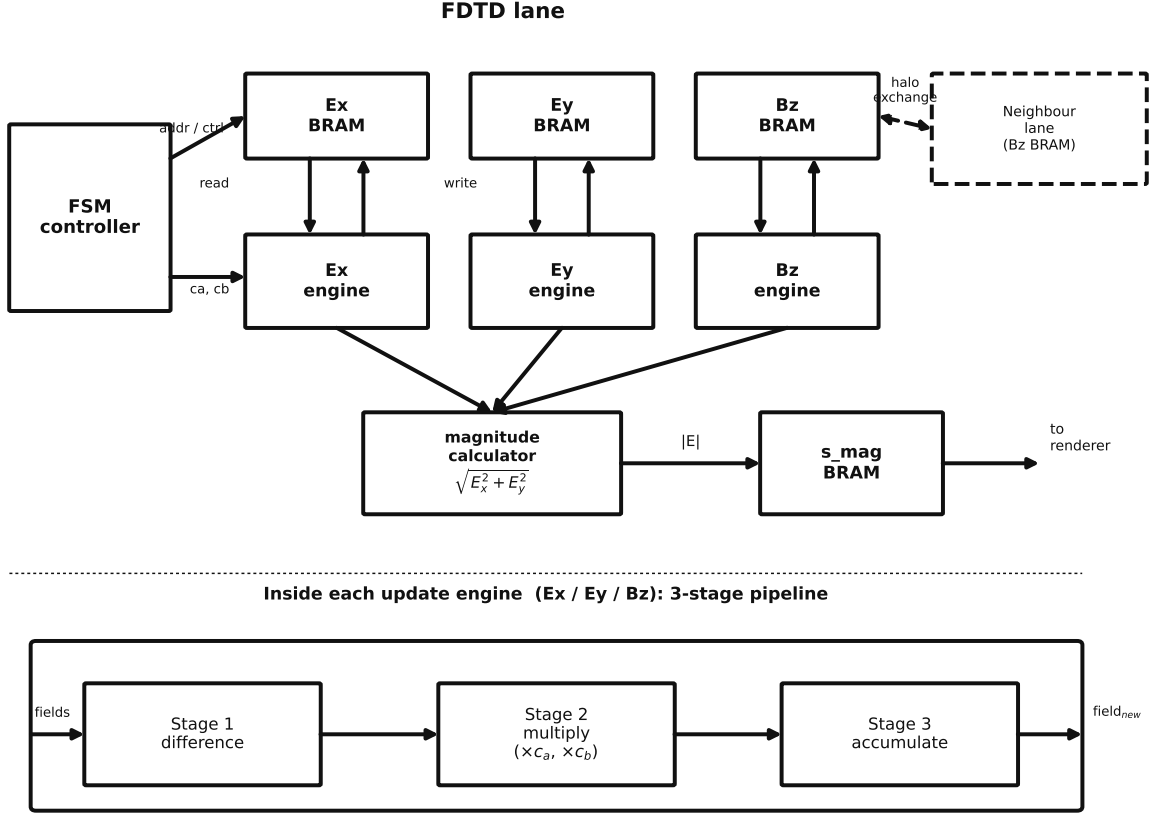


Figure 3.3: Datapath of a single FDTD lane. The FSM controller addresses the three field BRAMs ( $E_x$ ,  $E_y$ ,  $B_z$ ), each read by its own update engine and written back in place; a magnitude calculator reads the fields and writes  $|E|$  into the s\_mag BRAM for the renderer, and edge rows are exchanged with the neighbouring lane through the halo ports. Each update engine is the 3-stage pipeline shown below.

This equation is computed in the PL using the following 3 step pipeline:

Stage 1:

$$\delta = B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j] \quad (3.35)$$

Stage 2:

$$\alpha = c_a \cdot E_x^n[i, j], \quad \beta = c_b \cdot \delta \quad (3.36)$$

Stage 3:

$$E_x^{n+1}[i, j] = \alpha - \beta \quad (3.37)$$

The computation was broken down into a pipeline as it formed a critical path that failed timing in implementation; it still allowed us to have a throughput of 1-2 (depending on the pass) cell per cycle whilst ensuring timing requirements were met. Because each result is written 4 cycles

after its read is issued (1 cycle of BRAM read latency plus the 3 pipeline stages), the pipeline must also be drained at the end of each pass: the last 4 results only emerge after the final read, so the solver runs 4 cycles past the cell count to flush them. This makes an iteration  $2 \times \text{ROWS} \times \text{COLUMNS} + 4$  cycles, and without this drain the last 4 cells are never written.

### 3.9 Boundaries and the Absorbing Layer

An issue occurs at the boundary walls of our grid: as there are no cells beyond them, we would intuitively force them to zero to prevent the accessing of invalid indices. However due to the nature of derivatives this would lead to a reflection of the wave off the boundaries back into the grid as if they were walls, meaning within a few dozen frames the entire grid will become a superposing interfering mess and we would not be able to visualise any interesting experiments.

The industry method to simulate an "infinite plane" to prevent reflections is to use a method called the perfectly matched layer. This method gradually decays the wave near the boundaries to cause negligible reflection. This is done using multiple methods from convolution (CPML) to decay constants that scale linearly or polynomially (UPML). We chose to go with UPML due to its simplicity and accuracy while conserving memory resources, DSPs, LUT usage and cycles. We chose a cubic profile achieving  $-113$  dB as it lead to no visual reflection artifacts.

Cubic conductivity profile with depth  $d$  into the absorbing layer:

$$\sigma(d) = \sigma_{\max} \left( \frac{d}{d_{\max}} \right)^3 \quad (3.38)$$

Attenuation coefficient derived from the conductivity:

$$c_a(d) = \frac{2\varepsilon - \sigma(d) \Delta t}{2\varepsilon + \sigma(d) \Delta t} \quad (3.39)$$

At  $d = 0$  (interior):  $\sigma = 0$ , so  $c_a = 1$  and the field is unattenuated. The modified update inside the PML:

$$E_x^{n+1}[i, j] = c_a(d) \cdot E_x^n[i, j] - c_b \left( B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i-1, j] \right) \quad (3.40)$$

One-way attenuation after traversing the full PML, where the wave spends  $\approx 1/S$  iterations in each cell:

$$A = \prod_{d=0}^{d_{\max}} c_a(d)^{1/S} \quad (3.41)$$

A PML depth of 6 cells was chosen, after experimentation with different depths, because it was the smallest depth that led to unnoticeable reflections while keeping  $c_a$  large enough that the PML boundary itself wasn't acting as a wall. Due to the narrow depth of our PML boundaries we chose to store constants in registers (only 18) instead of BRAM as it traded minimal LUT usage for more BRAM availability and simpler logic, especially because we did not need the 1 cycle delay for reading and writing.

| Depth $d$ | $c_a$ | $c_a$ (real) |
|-----------|-------|--------------|
| 0         | 8192  | 1.0000       |
| 1         | 8174  | 0.9978       |
| 2         | 8045  | 0.9821       |
| 3         | 7695  | 0.9393       |
| 4         | 7014  | 0.8562       |
| 5         | 5892  | 0.7192       |

Table 3.4: Cubic attenuation ramp  $c_a \approx 8192 - 18.4 d^3$

Although the deepest PML cell only multiplies the field by  $c_a = 0.7192$ , a 28% reduction, this factor is applied on every iteration the wave spends in that cell. Because the wave moves at  $S = |c_b|/2^{13} \approx 0.0875$  cells per iteration it takes roughly  $1/S \approx 11$  iterations to cross a single cell, so the effective attenuation in that cell alone is  $0.7192^{11} \approx 0.02$ . Across all six cells of the layer the one-way attenuation is roughly  $-56$  dB, and since the wave reflects off the zeroed outer wall and passes back through, the round-trip attenuation is  $-113$  dB.

### 3.10 From Three Passes to Two

We chose to parallelise the computation of the Ex and Ey fields reducing our system from 3 pass (Ex, Ey and Bz) to 2 pass (E and B). This was because both Ex and Ey fields only varied on Bz which would have been computed entirely in the pass before and because of our sequential calculation of left to right for Ex and up to down for Ey we could use just 2 read port BRAM with 2 temporary registers to hold the previous Bz values to compute the next Ex and Ey values. This works because computing the new Ex requires Bz[i, j], Bz[i - 1, j] and computing the new Ey requires Bz[i, j], Bz[i, j - 1]. While it might seem that we need 3 read ports the Bz[i, j] value was actually used in the previous calculation so it can just be temporarily held in a register allowing us to make just 2 reads with the 2 read ports available on the single BRAM18 holding the Bz values. For initial conditions [0,0] we assume negative indices are just 0 instead of trying to read from an invalid address which still allows us to only use 2 BRAM ports. This allows us to parallelise Ex and Ey computation without making redundant copies of the BRAM and get an exact 50% performance gain.

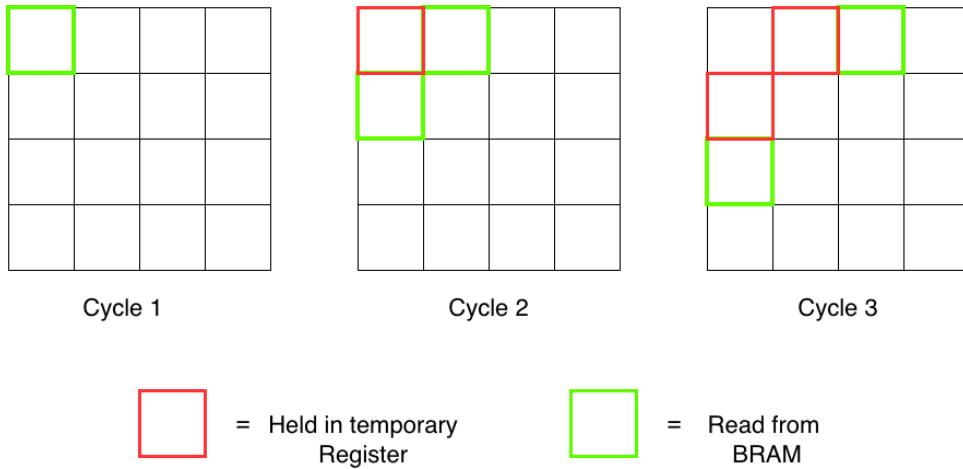


Figure 3.4: Example field updates showing that just 2 temporary registers are enough to make at most 2 new  $B_z$  reads per cycle.



### 3.11 Lane Parallelism and the Halo

Another optimisation was splitting the computation into multiple lanes running in parallel, each covering a different section of the Yee lattice. The simplest way to do this was splitting into horizontal strips, although vertical strips would have been equally suitable. A small halo exchange is needed: to compute the first row of the  $(n + 1)$ th strip, the last values of the  $n$ th strip for  $B_z$  and  $E_y$  are required, ensuring the wave propagates between strips.

This works with our limitation of 2 BRAM ports per lane per field. If we take the example of the first and second lane with the first row computation, because all lanes compute together, when the 2nd lane is on its first row the 1st lane is also on its first row, meaning the 1st lane is not using its own 2nd BRAM port as we assume negative indices are 0. This allows the 2nd lane to use the free BRAM port of the 1st lane to read the last row of the 1st lane's strip to "carry" over the wave. This works for the 2nd and 3rd lanes as well as the 2nd lane's own 2nd BRAM port is also free at that moment, allowing the third lane to use it to read the last row in the 2nd lane's strip. This allows us to forgo any stall or redundant copies logic and leads to a direct scaling between number of lanes and increase in performance.

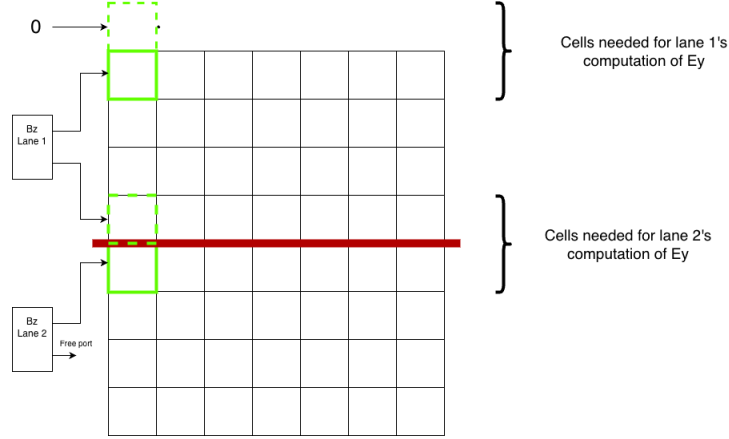


Figure 3.5: Example field update for  $E_y$  showing Halo exchange with 2 lane architecture. Solid lines are the 2 cells being updated and dashed line are  $B_z[i, j-1]$ . The free port can be used for the next lane.

Theoretically we could instantiate as many lanes as we wished and performance would scale linearly; however, this leads to BRAM segmentation. We had to keep in mind that the BRAM on the PYNQ-Z1 can only be partitioned down to BRAM18s, so if we had very small lanes we would have many partially filled BRAM18s whose ports were constantly under load from the FDTD solver, not allowing them to use the rest of the memory for other purposes. We therefore calculated that for a  $128 \times 128$  grid, the smallest column strip that fits in one whole BRAM18 was a  $128 \times 8$  strip. Another restriction was simply the wave speed: if we computed too many iterations per frame the wave would move too quickly, as there would be more iterations per frame; to combat this we were able to tune the constants in our PML submodule.

This meant that the highest-performance design we implemented was a 16-lane architecture, maximising the performance-to-memory trade-off. This gave a  $16\times$  speedup over our single-lane design, and characterised on its own at 100 MHz it reaches a throughput of 2.4 billion field updates per second. This 100 MHz, 16-lane figure characterises the solver architecture in isolation; the integrated, human-watchable demonstration instead runs four lanes at the 25 MHz field-datapath clock, because the full 16-lane rate moves the wave far too quickly to observe.

The figures in this section therefore describe the peak architecture rather than the slowed-down demonstration build.

This would result in the fields only taking up 24 BRAM36s out of 140, along with a further 5 for calculating the 16-bit field magnitude  $|S|$  or  $|E|$ , which is reduced to the renderer’s 8-bit heightmap, leaving the remaining 111 BRAM36s for any double buffering or data copies for the rendering engine.

|                             | 1 lane      | 4 lane      | 16 lane     |
|-----------------------------|-------------|-------------|-------------|
| Grid                        | 64×64       | 64×64       | 128×128     |
| Iteration latency           | 8196 cycles | 2052 cycles | 2052 cycles |
| Iterations per 60 fps frame | 203         | 812         | 812         |
| Field updates /s            | 150 Million | 600 Million | 2.4 Billion |

Table 3.5: Solver performance scaling with lane count at 100 MHz

### 3.12 Performance Comparison

As shown in Table 3.5, our 16-lane FDTD solver has a throughput of 2.4 billion field updates per second. To get a reference of how fast that is, we tested how many field updates the PYNQ’s A9 CPU would compute in the same amount of time. As shown in Figure 3.6, there was a 31.7× speedup in the performance of the FDTD algorithm, even against an optimised C implementation on the A9 CPU. Furthermore as shown in Table 3.6 the PL is 96 times more power efficient.

| config        | field updates | vs fpga |
|---------------|---------------|---------|
| 1 core scalar | 29.8 M/s      | 80.5x   |
| 2 core scalar | 58.3 M/s      | 41.2x   |
| 2 core + NEON | 75.7 M/s      | 31.7x   |
| fpga 16 lane  | 2400.0 M/s    | 1.0x    |

Figure 3.6: Solver throughput on the Cortex-A9 against the 16 lane FPGA design

|                           | Throughput | Power  | Efficiency  |
|---------------------------|------------|--------|-------------|
| Cortex-A9, 2 cores + NEON | 75.7 M/s   | ~1.5 W | ~50 M/s/W   |
| FPGA 16 lane solver       | 2400 M/s   | ~0.5 W | ~4800 M/s/W |

Table 3.6: Field update throughput per watt. CPU throughput measured on the PYNQ-Z1. Power figures are the estimated PS load power and Vivado power estimate for the PL respectively

### 3.13 Verification

Figures 3.7–3.10 show the results of our unit test benches. We prioritised thorough test benches with comprehensive test cases to ensure that, when implementing our designs in Vivado, there would be minimal issues. We made test benches for all the core modules and then top level test benches for the entire FDTD system for 4 and 16 lane to ensure the systems subparts worked before verifying the overall system worked.

```

test 1 passed: d=0 ca=8192 cb=-717
test 2 passed: d=1 ca=8174 cb=-717
test 3 passed: d=2 ca=8045 cb=-717
test 4 passed: d=3 ca=7695 cb=-717
test 5 passed: d=4 ca=7014 cb=-717
test 6 passed: d=5 ca=5892 cb=-717
test 7 passed: d=6 default ca=8192 cb=-717
test 8 passed: ca ramp monotone with depth
all 8 tests passed
tests/tb_pml.sv:102: $finish called at 7000 (1ps)

```

Figure 3.7: PML module tests

```

test 1 passed: iteration took 2052 cycles
test 2 passed: source injected
test 3 passed: top boundary clear
test 4 passed: bottom boundary clear
test 5 passed: left boundary clear
test 6 passed: right boundary clear
test 7 passed: wave crossed lane seam
test 8 passed: done refires
test 9 passed: reset clears state
all 9 tests passed
tests/tb_top_fdt_d_quad_lane.sv:186: $finish called at 453085000 (1ps)

```

Figure 3.9: 4 lane system tests

```

test 1 passed: iteration took 32772 cycles
test 2 passed: source injected, ey = 8192
test 3 passed: ey boundary rows clear
test 4 passed: ex boundary cols clear
test 5 passed: done refires
test 6 passed: reset halts solver
test 7 passed: pml attenuates, interior untouched (pml=7695 interior=8192)
all 7 tests passed
tests/tb_fdt_d_solver.sv:248: $finish called at 984415000 (1ps)

```

Figure 3.8: Single solver tests, 128×128

```

test 1 passed: iteration took 2052 cycles
test 2 passed: source injected
test 3 passed: top boundary clear
test 4 passed: bottom boundary clear
test 5 passed: left boundary clear
test 6 passed: right boundary clear
test 7 passed: wave crossed lane seam
test 8 passed: done refires
test 9 passed: reset clears state
all 9 tests passed
tests/tb_top_fdt_hex_lane.sv:216: $finish called at 453085000 (1ps)

```

Figure 3.10: 16 lane system tests

# Chapter 4

## The Renderer

### 4.1 Foundations and the baseline design

#### 4.1.1 The rendering problem

The renderer’s task is to turn a two-dimensional field produced by the solver into a shaded three-dimensional image, in real time, on an HDMI display. The field is stored as a height-field surface: a grid of values  $h(x, y)$  giving the surface elevation at each cell. The renderer casts one ray per screen pixel into this surface and returns the colour that ray sees, so the image is a pure function of pixel coordinates,

$$\text{colour} = f(x, y),$$

evaluated in raster order. Because every pixel’s ray is computed entirely independently of every other, the workload is embarrassingly parallel: there is no shared state between pixels, no feedback, and no data dependency across the frame.

#### 4.1.2 The fixed pixel pipeline

Every design in this chapter, from the baseline to the final parallel renderer, is the same four-stage pixel pipeline. What changes between generations is how memory is scheduled and how many pixels are in flight, never the underlying algorithm. The four stages are:

| Stage          | Function                                                                                                                                           |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ray_gen</b> | Converts a screen pixel $(x, y)$ into a world-space ray direction $D$ from the camera basis.                                                       |
| <b>marcher</b> | Chains $N$ march steps; each advances the ray, looks up the heightmap, and tests for a surface crossing. Outputs the hit cell and hit/miss status. |
| <b>normal</b>  | Estimates the surface normal at the hit from the local height gradient.                                                                            |
| <b>shader</b>  | Applies diffuse lighting and height colouring to produce $(R, G, B)$ , or sky colour on a miss.                                                    |

Table 4.1: The four stages of the fixed pixel pipeline.

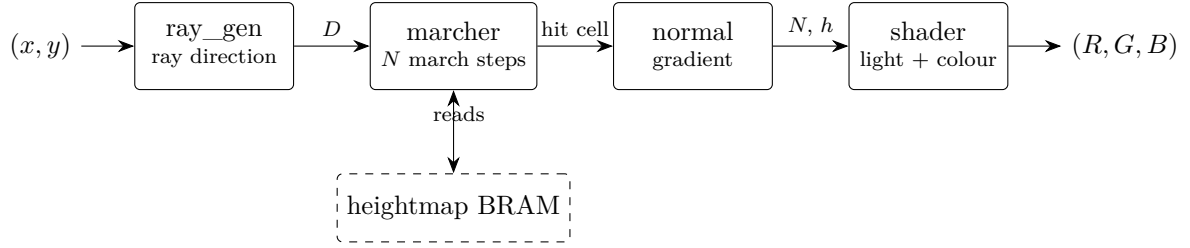


Figure 4.1: The fixed four-stage pixel pipeline, common to every design in this chapter. Later generations modify only the memory scheduling inside **marcher** and the number of pipelines in parallel.

### 4.1.3 Number representation

The renderer uses fixed-point arithmetic throughout; no floating-point is synthesised. The formats are summarised in Table 4.2. Word length is a substantial design choice: more fractional bits give a smoother surface and fewer staircase artefacts, but widen every multiplier (more DSP) and tighten timing, while more integer bits enlarge the representable world at the cost of precision. The chosen 16-bit **Q3.13** formats place the world in  $[-4, 4)$  units with a resolution of  $\approx 1.2 \times 10^{-4}$ , which is fine enough that the surface looks continuous once interpolation is added (Section 4.2).

| Signal                 | Format | Bits | Role                              |
|------------------------|--------|------|-----------------------------------|
| Position $P$           | Q3.13  | 16   | ray world coordinate              |
| Direction $D$          | Q3.13  | 16   | ray / basis vector                |
| Height $h$             | Q3.13  | 16   | heightmap value (stored as 8-bit) |
| Screen UV $u, v$       | Q1.14  | 16   | centred pixel coordinate          |
| Per-pixel constant $K$ | Q0.15  | 16   | screen-scale factor (= 137)       |

Table 4.2: Fixed-point formats used across the renderer. All are signed.

### 4.1.4 Choosing the march depth

Before the architecture can be sized, the march depth must be fixed, because it is the dominant memory cost: each step adds heightmap read ports, and every two reads landing in the same cycle force another dual-port copy of the entire heightmap. Depth is therefore bounded above by the BRAM budget, and the right choice is the smallest number of steps  $N$  that still captures essentially every surface hit, any step beyond that spends memory for no visual gain.

To locate that floor, the ray generator and march loop were reproduced in fixed-point in Python using the exact **Q3.13** formats, the same camera basis, and the same surface-crossing test as the RTL and run over a representative field surface (two source poles with propagating ripples, matching the solver output). For every pixel, the script records the step index at which its ray first hits the surface. The resulting distribution is shown in Figure 4.2 and the cumulative coverage in Table 4.3.

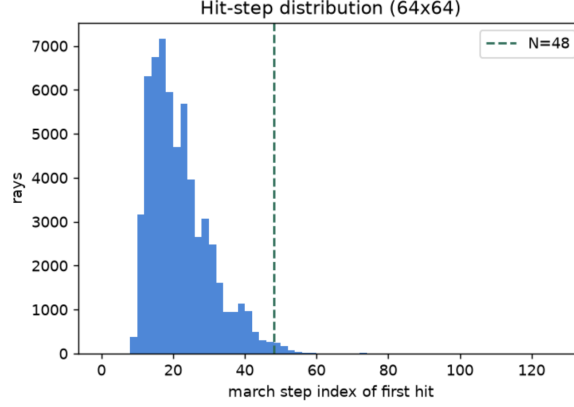


Figure 4.2: Step index at which each ray first hits the surface ( $64 \times 64$  grid, fixed-point march identical to the RTL). The distribution rises near step 10, peaks around step 17, and its tail vanishes by step 48.

| $N$ steps | Hits captured |
|-----------|---------------|
| 16        | 33.9%         |
| 24        | 71.0%         |
| 32        | 89.3%         |
| 40        | 96.5%         |
| <b>48</b> | <b>99.2%</b>  |
| 64        | 100.0%        |

Table 4.3: Cumulative fraction of surface hits captured by march depth  $N$ , over the representative field at  $64 \times 64$ .

Coverage climbs steeply through the knee of the curve and reaches 99.2% at 48 steps; the final 0.8% would cost a further 16 steps and the BRAM that comes with them. The march depth is therefore set to 48 steps for the  $64 \times 64$  grid, the smallest depth at full coverage.

One detail of the step arithmetic is chosen specifically to save DSPs. Each step advances the ray by  $P += DT \cdot D$ , a multiply by the step size  $DT$  on every axis. By fixing  $DT$  to a power of two ( $DT = 256 = 2^8$ ), this multiply becomes a simple bit-shift, costing no DSP at all. With  $DT$  an arbitrary value, each step would need three multipliers for the position advance :  $48 \times 3 = 144$  extra DSP across the marcher, so the power-of-two choice leaves only the interpolation itself as the per-step multiplier cost.

#### 4.1.5 The baseline architecture

With the depth fixed, the baseline establishes the skeleton on which every later design is built. It marches 48 steps over a  $64 \times 64$  heightmap, running at one pixel per clock cycle in a single clock domain locked to the pixel clock.

Each march step issues one heightmap read: it samples the single nearest grid cell, so 48 steps need 48 read ports. The normal stage reads four neighbouring cells to form a central-difference gradient, adding four more. The baseline therefore exposes  $48 + 4 = 52$  read ports. A Xilinx **RAMB36** block is dual-port, serving two reads per cycle, so these map to **26 BRAM tiles**. The pipeline latency is  $4 + (48 \times 4) + 4 + 1 = 201$  cycles, after which it sustains one finished pixel every cycle.

At 26 tiles this baseline already consumes  $26/140 = 18.6\%$  of the device’s BRAM budget, before any interpolation is added.

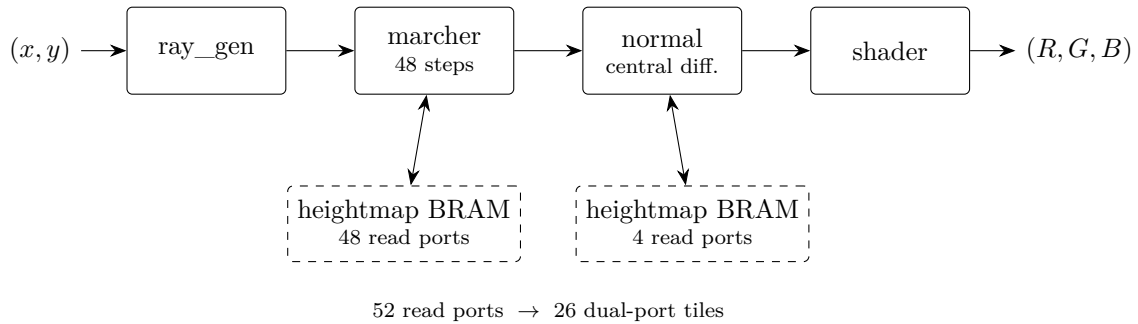


Figure 4.3: Baseline renderer. Forty-eight marcher ports plus four central-difference normal ports give 52 read ports (26 BRAM tiles).

Because hit detection and shading both use the single nearest sample, the baseline image is visibly blocky (Figure 4.4). Removing that blockiness is the subject of the next two sections; the baseline’s role is to fix the interfaces, the fixed-point formats, and the pipeline timing that every later design inherits unchanged.

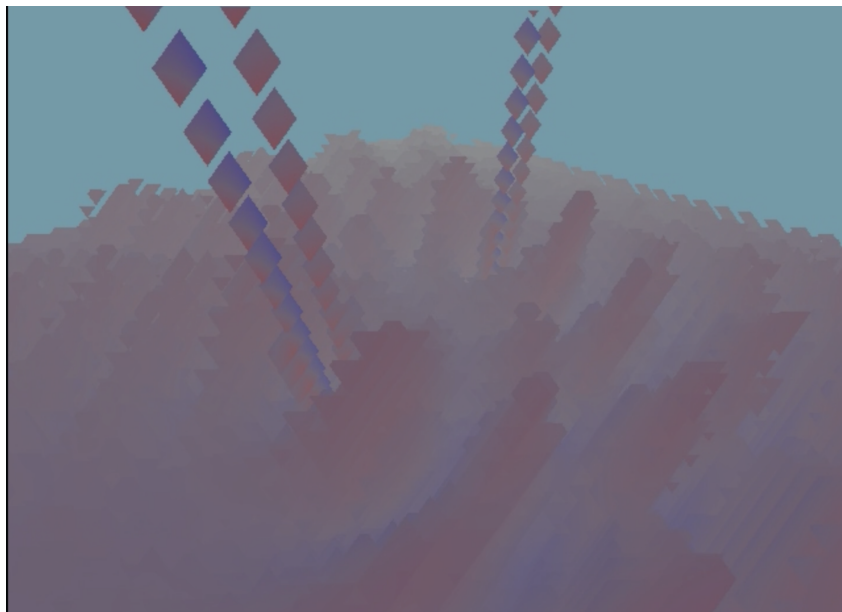


Figure 4.4: Baseline output. The nearest-sample surface renders as discrete plateaus with hard cell-boundary edges that motivate interpolation.

## 4.2 Shading interpolation

The baseline’s blockiness has two separate causes, and they are worth separating because they have very different costs to fix. The silhouette, where terrain meets sky, is blocky because the marcher stops on whole cells. The shading, the lighting and colour across each visible face, is blocky because the normal and the surface height are taken from a single nearest cell, so every cell renders as one flat-shaded plateau. This section fixes the second cause, and does so at zero additional BRAM, which is why it comes first.

### 4.2.1 Reusing the reads the normal already makes

The insight is that the normal stage already pays for four heightmap reads, and those four reads contain everything a smooth surface needs. In the baseline the normal reads four neighbours in a “plus” pattern  $(ix \pm 1, iy \pm 1)$  to form a central difference. If instead it reads the four corners of the cell the ray landed in,

$$h_{00} = (ix, iy), \quad h_{10} = (ix+1, iy), \quad h_{01} = (ix, iy+1), \quad h_{11} = (ix+1, iy+1),$$

then the same four values serve two purposes at once: a forward-difference normal and a bilinearly-interpolated surface height. The read count is unchanged, still four ports, still the same two BRAM tiles, so the improvement is free in memory.

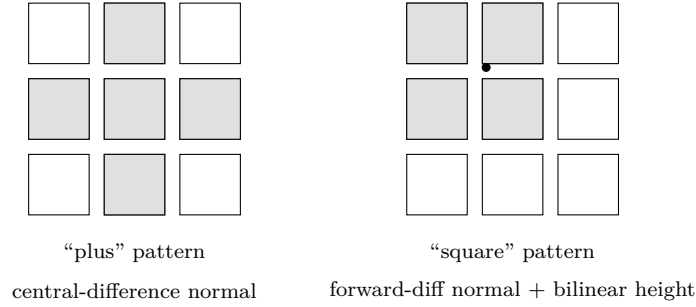


Figure 4.5: The baseline reads four axis neighbours for a central-difference normal (left). Design 1 reads the four corners of the hit cell instead (right), serving both a forward-difference normal and a bilinear surface height from the same four reads and no extra ports.

### 4.2.2 The interpolation mathematics

The primitive underneath everything here is linear interpolation (a “lerp”): given two corner heights and a fractional position between them, estimate the height on the straight line joining them. Bilinear interpolation extends this to a cell by applying it three times, two lerps along the top and bottom edges in  $x$ , then one lerp between those results in  $y$ , recovering a smooth surface height  $h_{\text{interp}}$  from the four corners  $h_{00}, h_{10}, h_{01}, h_{11}$  at the ray’s sub-cell position  $(x_f, y_f)$ . Figure 4.6 shows the construction.

Each lerp is one subtract, one multiply by the 8-bit fraction (a single DSP slice), and one add, kept entirely in fixed point.

**Parallelism:** On a CPU these operations are inherently sequential: the two  $x$ -lerps and the forward differences would issue one after another, and the final  $y$ -lerp would wait on both  $x$ -lerps, costing several dependent instructions per pixel. In the FPGA the computation is laid out spatially instead of temporally. The two  $x$ -lerps  $h_{\text{top}}$  and  $h_{\text{bot}}$  are independent, so they are placed in the same pipeline stage and their multipliers run in the same clock cycle; the forward differences  $N_x$  and  $N_y$  are computed in that same stage on their own adders.



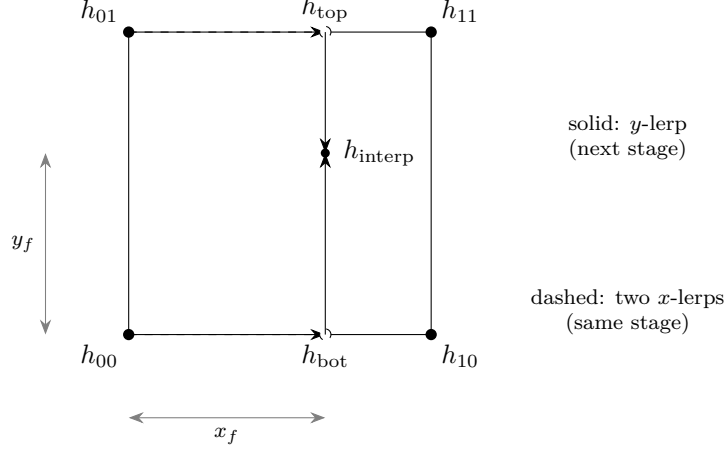
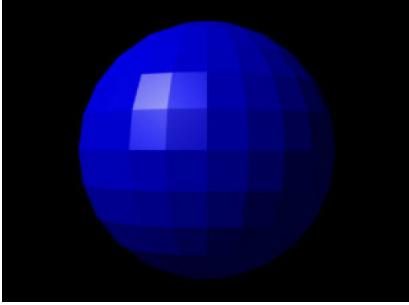


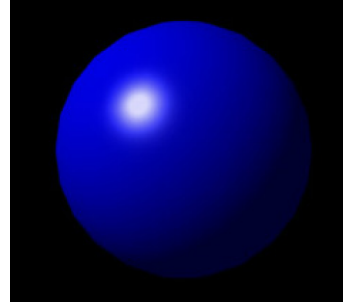
Figure 4.6: Bilinear interpolation as three lerps over the hit cell: two edge lerps in  $x$  give  $h_{\text{top}}$  and  $h_{\text{bot}}$ , and a final lerp in  $y$  gives the smooth surface height  $h_{\text{interp}}$ .

#### 4.2.3 Cost

The change adds one pipeline stage to the normal, the final  $y$ -lerp takes its latency from four cycles to five, and a small amount of logic: three lerp multipliers and two forward-difference subtractions. No BRAM-heavy structure is introduced, and the  $48 + 4 = 52$  read ports (**26 tiles**,  $26/140 = 18.6\%$ ) are unchanged from the baseline.



(a) Flat (nearest-sample) shading.



(b) Design 1 bilinear shading.

Figure 4.7: Flat-shaded baseline (left) versus Design 1 shading interpolation (right). The per-cell facets disappear and the lighting varies smoothly across the surface; the silhouette is unchanged.

### 4.3 Folding and the FIFO scanout

Folding the renderer to save BRAM has a consequence beyond throughput: it changes the clock domain the renderer must live in. At one pixel per cycle the renderer could be clocked directly at the display's pixel rate, but a folded renderer produces pixels more slowly, so to keep the display fed it must run in a faster clock domain than the HDMI output. The two domains must then be bridged. This section describes the video pipeline that performs that bridge, the clocking, the timing generation, and the asynchronous FIFO that carries pixels across the domain boundary, together with the reliability measures that keep the display locked.

#### 4.3.1 Clocking and the two domains

The HDMI output was implemented as a reusable PYNQ-Z1 video pipeline around the ray-rendering core. The board's 125 MHz PL clock is converted using a Vivado Clocking Wizard into a 25 MHz pixel clock for  $640 \times 480$  VGA timing, a 125 MHz serial clock for TMDS generation, and a 100 MHz renderer clock. A timing module generates the full  $800 \times 525$  VGA raster, including blanking, HSYNC, VSYNC, and active-video enable. The renderer operates in the 100 MHz domain, while HDMI scan-out operates in the 25 MHz domain, so rendered RGB pixels cross through an asynchronous FIFO before being passed to the Digilent `rgb2dvi` IP, which drives the differential HDMI TMDS signals.

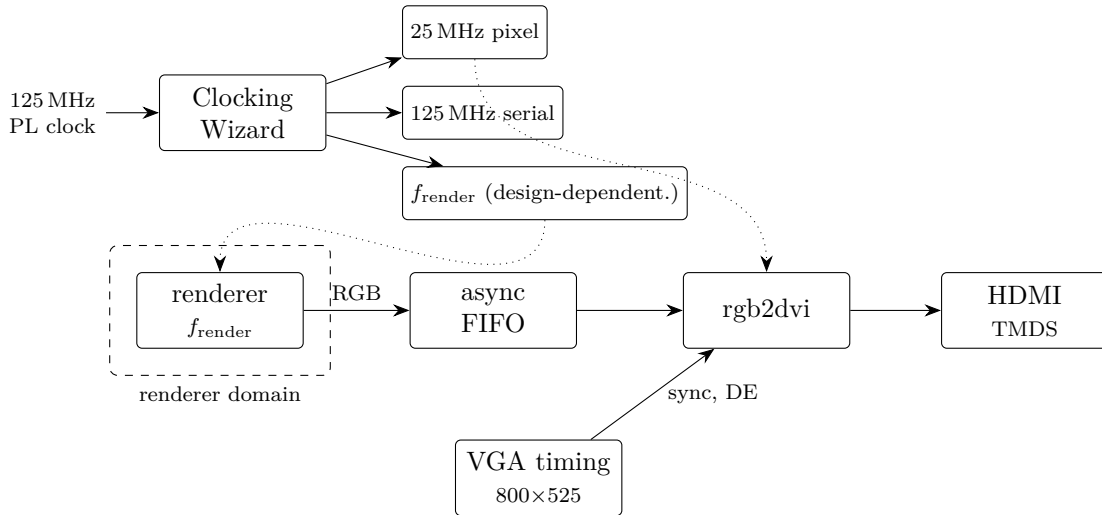


Figure 4.8: The HDMI video pipeline. The Clocking Wizard derives the 25 MHz pixel and 125 MHz serial clocks from the 125 MHz PL clock, plus a renderer clock  $f_{\text{renderer}}$  whose value depends on the design's fold factor. Rendered pixels cross an asynchronous FIFO from the renderer domain into the 25 MHz pixel domain before `rgb2dvi` drives the TMDS outputs.

#### 4.3.2 Matching throughput and latency to the display

Different renderer versions can use the same HDMI pipeline as long as their throughput matches the fixed display rate. HDMI requires one pixel every 40 ns during active video; for example, the 48-step Design 4 renderer produces one pixel every four 100 MHz cycles, matching this rate. Pipeline latency is handled by delaying HSYNC, VSYNC, and data-enable until they align with the corresponding RGB result, while throughput is handled by the renderer schedule and FIFO. The FIFO absorbs clock-domain phase differences and short stalls, but it cannot compensate for a renderer that is consistently slower than the HDMI pixel rate. Therefore, each design must be checked using both latency and initiation interval: latency determines the required video-control delay, and initiation interval determines whether active video can be continuously filled.

### 4.3.3 Reliability

A reliability fix was added so `vid_pVDE` remains tied to display timing rather than FIFO availability; if the FIFO underflows, magenta diagnostic pixels are shown instead of dropping HDMI signal lock.

### 4.3.4 The limit of the FIFO scanout

The FIFO works only while the renderer can be clocked fast enough to deliver one pixel every 40 ns of active video on average. For a renderer folded to one pixel every  $F$  cycles, sustaining the 25 Mpix/s active rate requires a PL clock of

$$f_{\text{PL}} = F \times 25 \text{ MHz},$$

since each pixel now takes  $F$  clock cycles instead of one. Table 4.4 evaluates this for each fold factor against the 125 MHz limit.

| Fold $F$ | Throughput    | Required $f_{\text{PL}}$ | Feasible ( $\leq 125 \text{ MHz}$ )? |
|----------|---------------|--------------------------|--------------------------------------|
| 1        | 1 px / cycle  | 25 MHz                   | yes                                  |
| 2        | 1 px / 2 cyc  | 50 MHz                   | yes                                  |
| 4        | 1 px / 4 cyc  | 100 MHz                  | yes                                  |
| 8        | 1 px / 8 cyc  | 200 MHz                  | no                                   |
| 16       | 1 px / 16 cyc | 400 MHz                  | no                                   |

Table 4.4: PL clock required to sustain the 25 Mpix/s active display rate for each fold factor, with a direct FIFO scanout. Beyond a four-cycle fold the required clock exceeds the device’s limit.

At one pixel every four cycles the required 100 MHz is met; this is exactly the clock at which Design 4 runs. Folding any further, however, demands 200 MHz at eight cycles and 400 MHz at sixteen, both well beyond the 125 MHz the PL can support. The FIFO scanout is therefore hard-limited to a four-cycle fold: it cannot exploit the deeper folding that would save the most BRAM, because there is no clock fast enough to keep the display fed. Breaking this limit requires decoupling the renderer from the display entirely, which is the subject of Section 4.5.

## 4.4 Silhouette interpolation and the folded board design

Shading interpolation smoothed the lit surface but left the silhouette, the boundary between terrain and sky, on cell edges, because the marcher still stops at the nearest whole cell. Smoothing that boundary is the one improvement that genuinely cannot be made for free: it requires the marcher itself to interpolate at every step, not just once at the hit.

### 4.4.1 Bilinear in the marcher, and the cost explosion

To make the ray stop on the smooth surface rather than on a cell boundary, each march step must test the ray against the bilinearly-interpolated height, not the nearest sample. That means every step now needs the four corners of the cell it is in: four heightmap reads per step instead of one.

The consequence for memory is severe. With four corner reads at every step, 48 steps perform  $48 \times 4 = 192$  heightmap reads per pixel. Design 3 spreads these over a two-cycle window (half-rate), giving  $192/2 = 96$  reads per cycle. A dual-port `RAMB36` serves two reads per cycle, so

this needs  $96/2 = 48$  heightmap copies; at  $64 \times 64 \times 16$ -bit each copy spans two tiles, for **96 tiles** ( $96/140 = \mathbf{68.6\%}$ ).

The renderer also does not have the full BRAM budget to itself, since the solver team’s FDTD engine also consumes RAMB36 tiles for its field storage, so the renderer’s true ceiling is well below 140. This makes the cost decisive, scaling the heightmap to  $128 \times 128$ , which would render the propagating wave far more clearly, multiplies each copy’s depth from two tiles to eight and is simply not feasible in this architecture.

#### 4.4.2 Folding the reads

The following upgrade changes nothing about the mathematics or the image; it changes only when the reads happen. Instead of four corner reads over two cycles on two ports, each step issues its four reads one at a time over four cycles on a single port:

$$\text{Previous Design: } \frac{4 \text{ reads}}{2 \text{ cycles}} \times 2 \text{ ports} \longrightarrow \text{New Design: } \frac{4 \text{ reads}}{4 \text{ cycles}} \times 1 \text{ port.}$$

The total reads per pixel are identical, they are simply spread more thinly in time. Folding the same 192 reads over a four-cycle window gives  $192/4 = 48$  reads per cycle, served by  $48/2 = 24$  dual-port copies, or **48 tiles**. On top of this, this design forwards the four hit-cell corners the marcher has already read directly into the normal stage, so the normal no longer reads the heightmap at all and adds no tiles of its own. This design therefore renders the smooth silhouette in **48 tiles** ( $48/140 = 34.3\%$ ), with the saving bought purely through throughput: one pixel every four cycles instead of every two.

Even at one pixel every four cycles, the renderer still produces a frame faster than the display can show it: at 100 MHz this is 25 Mpix/s, or  $\sim 81$  fps at  $640 \times 480$ , comfortably above the 60 fps HDMI cap, so the throughput traded away for BRAM costs nothing in perceived performance.

#### 4.4.3 Folding the multipliers: from 70% to 26% DSP

The idle time that folding creates can be exploited a second time, on the multipliers. Each march step’s bilinear interpolation uses three multiplies; unrolled across 48 steps that is  $48 \times 3 = 144$  DSP, which with the ray-generation and shading stages totals roughly 150, about 70% of the device’s 220 DSP48E1 slices. This is the same kind of waste folding already removed from the memory: at one pixel every four cycles, each step’s multiplier is busy only one cycle in four and idle the other three.

The fix is identical in spirit to the BRAM fold. The three lerps of a step are cascaded into sub-stages that time-share a single DSP across the four-cycle window, so one multiplier does the work the three previously did. This collapses the marcher from 144 to 48 DSP. Including ray-generation and shading, the whole renderer now uses just **58 DSP** ( $58/220 = \mathbf{26.4\%}$ ), down from 70%.

|                 | Parallel multipliers | Time-shared multipliers |
|-----------------|----------------------|-------------------------|
| DSP utilisation | 70%                  | <b>26%</b>              |

Table 4.5: Vivado post-implementation DSP utilisation before and after folding the multipliers. All other resources are unchanged; DSP drops from 70% to 26% while BRAM holds at the predicted 35% (49/140 tiles).

The design closes timing at 100 MHz with positive setup and hold slack, as summarised in Table 4.6.

| Metric            | Value     |
|-------------------|-----------|
| Setup slack (WNS) | +0.457 ns |
| Hold slack (WHS)  | +0.016 ns |
| Core clock        | 100 MHz   |

Table 4.6: Design 4 timing closure on the PYNQ-Z1: positive setup and hold slack at 100 MHz.

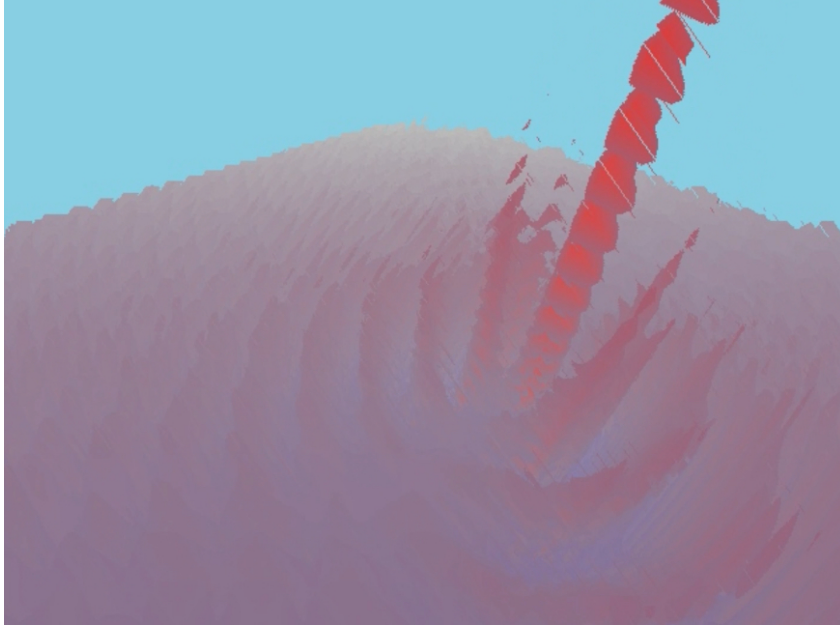


Figure 4.9: Output of the board-verified Design 4 renderer ( $64 \times 64$  field): the source pole and its propagating ripples are rendered with smooth shading and a smooth interpolated silhouette, on hardware at  $640 \times 480$ .

#### 4.4.4 Scaling to $128 \times 128$ is still not possible without parallel lanes

The same calculation rules out scaling to  $128 \times 128$ , however. A  $128 \times 128 \times 16$ -bit heightmap is 256 Kibit, so each copy now spans  $\lceil 256/36 \rceil = 8$  tiles instead of two. Keeping 48 steps and the four-cycle fold still needs 24 copies, but at eight tiles each that is  $24 \times 8 = \mathbf{192}$  tiles ( $192/140 = 137\%$ ), and the solver’s own BRAM pushes the true ceiling lower still. Even Design 4’s folding therefore cannot reach the resolution that would show the propagating wave clearly, which is the limit the next sections set out to break.

### 4.5 The frame buffer and parallel lanes

The natural next goal is to scale to a larger heightmap, so the propagating wave is shown in far greater detail. The obstacle is not the renderer’s logic but its output path. With the direct-to-HDMI scheme, the renderer must deliver one active pixel every pixel-clock to keep the display fed, which ties its throughput rigidly to the 25 Mpix/s active rate. A larger, deeper render needs more cycles per pixel, and to still sustain 25 Mpix/s the renderer clock would have to rise in proportion; the deeper folds of 1 pixel every 8 cycles and 1 pixel every 16 cycles require clocks running at 200 MHz and 400 MHz respectively, both far beyond the xc7z020’s 125 MHz support limit. The direct path simply cannot clock fast enough to scale.

The way out is to break the rigid link between render rate and display rate. This section does so with two changes: decouple the renderer from the display using a frame buffer, then recover

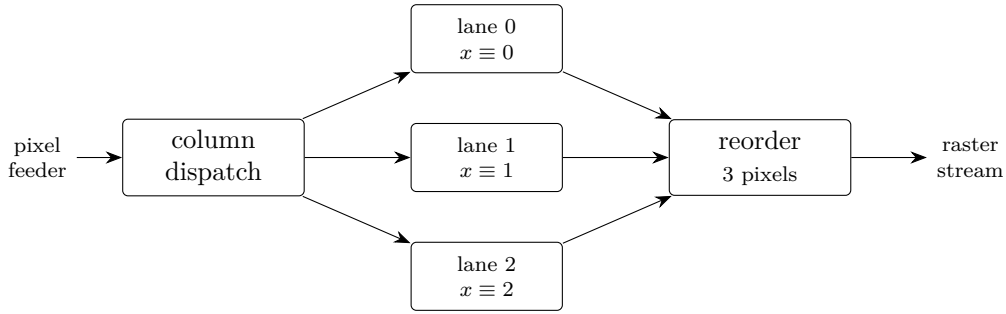


The remedy is parallelism: we run several independent lanes at that deep fold so their combined rate reaches 60 fps while each lane stays cheap.

This works because the rendering problem is embarrassingly parallel: every pixel is independent, so the screen can simply be divided between lanes. Three lanes are used, dispatched by column: lane  $L$  renders every third column ( $x \equiv L \bmod 3$ ). The three lanes are identical pipelines fed the same column-triple each period, so their outputs emerge aligned and re-serialise into raster order through a small three-pixel reorder buffer. The aggregate rate is

$$3 \times \frac{1 \text{ px}}{16 \text{ cyc}} \times 100 \text{ MHz} = 18.75 \text{ Mpix/s} \approx 61 \text{ fps at } 640 \times 480,$$

back above the 60 fps cap.



lane  $L$  renders columns  $x \equiv L \bmod 3$ , each at a 16-cycle fold

Figure 4.12: Three column-interleaved lanes (lane  $L$  renders  $x \equiv L \bmod 3$ ) running at a sixteen-cycle fold. Their aligned outputs re-serialise to raster order through a three-pixel reorder buffer, reaching  $\sim 61$  fps.

### 4.5.3 Why this uses fewer resources

The decisive point is that three deeply-folded lanes cost less memory than one lightly-folded lane, not more. A single lane at the four-cycle fold spreads its reads over four cycles and needs a certain number of heightmap copies; folding sixteen cycles instead spreads the same reads four times more thinly, cutting each lane's copies by four. Three such lanes therefore use  $3 \times \frac{1}{4} = \frac{3}{4}$  of the single-lane copy count, fewer copies overall, while running at 60 fps through the frame buffer. Parallelism at a deep fold is strictly cheaper in BRAM than a single lane at a shallow fold.

Three deeply-folded lanes therefore use less BRAM than one shallow-folded lane, leaving budget to spend on a larger grid, more steps, and a smoother silhouette, which the next section explores.

## 4.6 Final comparison: five candidate designs

The frame buffer and parallel lanes remove the frame-rate ceiling, but they do not remove the resource ceiling. Scaling to  $128 \times 128$  raises a new problem: the step size DT scales with the grid, so at  $128 \times 128$  each step covers half the world distance it did at  $64 \times 64$ , and the ray needs roughly twice as many steps to reach the same distance. Rerunning the hit-coverage analysis of Section 4.1.4 at  $128 \times 128$  confirms this, 48 steps now cover only 87% of hits, and reaching the 99% knee would take about 80 steps. The design the project would most like to build, three lanes, 80 steps for full coverage, bilinear silhouette,  $128 \times 128$ , 16-bit heights, is far out of budget on both BRAM and DSP. Something must be traded away.

Each advantage developed in this chapter (parallel lanes, full hit coverage, the bilinear silhouette, the full  $128 \times 128$  grid, and 16-bit height precision) can be relaxed in turn, giving five candidate designs. They are compared in Table 4.7.

| Design                  | Lanes    | Steps     | Silhouette      | Grid                      | Height       | Tiles     | Coverage   |
|-------------------------|----------|-----------|-----------------|---------------------------|--------------|-----------|------------|
| 1 · fewer steps         | 3        | 48        | bilinear        | $128^2$                   | 16-bit       | 72        | 87%        |
| 2 · drop silhouette     | 3        | 64        | nearest         | $128^2$                   | 16-bit       | 48        | 97%        |
| 3 · small grid          | 3        | 64        | bilinear        | $64^2$                    | 16-bit       | 48        | 100%       |
| 4 · two-lane full       | 2        | 64        | bilinear        | $128^2$                   | 16-bit       | 128       | 97%        |
| <b>5 · 8-bit height</b> | <b>3</b> | <b>64</b> | <b>bilinear</b> | <b><math>128^2</math></b> | <b>8-bit</b> | <b>96</b> | <b>97%</b> |

Table 4.7: Five candidate designs. Each keeps four advantages and relaxes one. Tiles are total RAMB36 count; coverage is the fraction of surface hits captured at  $128 \times 128$ . Designs 1–3 and 5 run at 61 fps; design 4 at 41 fps.

**Design 5: halve the height precision.** The other four designs each give up something the eye notices, coverage, edge smoothness, detail, or frame rate. Design 5 instead attacks the cost at its source: the size of a heightmap entry. Storing each height in 8 bits rather than 16 shrinks a copy from 8 tiles to 4, so the full design (three lanes, 64 steps, bilinear silhouette,  $128 \times 128$ , 61 fps, 97% coverage) costs

$$3 \times 8 \times 4 = 96 \text{ tiles } (96/140 = 68.6\%),$$

and keeps every other advantage intact. Its price is height precision: 8 bits gives 256 levels rather than 65,536. This is the least visible of the five trades, and the rendered surface varies smoothly — the gap between two adjacent height levels is far below what the eye can resolve against the display’s own colour quantisation, which is why Design 5 is chosen for the final renderer.

#### 4.6.1 Fitting the DSP budget

BRAM is not the only resource three lanes multiply. Each march step’s bilinear interpolation, after the time-sharing of Section 4.4, uses one DSP, so a 64-step lane spends 64 DSP in its marcher; with ray-generation and shading the single lane would sit near 74 DSP, and three lanes at that rate would need 222 DSP just over the device’s 220.

The shading stage is optimised to close this gap. By fixing the sun at a constant direction, the diffuse lighting dot-product becomes a multiplication by constants rather than by runtime values; constant multipliers synthesise into shift-and-add logic in the fabric rather than DSP slices, cutting the per-lane ray-generation and shading overhead to 6 DSP. The whole renderer then uses

$$3 \times (64 + 6) = 210 \text{ DSP } (210/220 = 95.5\%),$$

fitting within budget.

#### 4.6.2 The final design

The final renderer is three lanes at 61 fps, a 64-step bilinear march over a  $128 \times 128$  field with 8-bit heights and a fixed sun, reaching 97% hit coverage. It occupies 96 BRAM tiles and 210 DSP slices. Together with the solver’s 29 tiles, this places 125 of the device’s 140 BRAM tiles (89.3%) and 210 of its 220 DSP slices (95.5%) in use, the design is fitted closely to the hardware it runs on.

#### 4.6.3 Renderer performance

As shown in Tables 4.8 and 4.9, the FPGA renderer achieves a sustained throughput of 60 frames per second while processing 18.43 Mpix/s and performing approximately 1,179.6 Msample/s of ray-marching computations. To provide a reference for this performance, equivalent rendering



workloads were evaluated on the PYNQ’s Cortex-A9 CPU. The FPGA renderer achieves a speedup of  $50.8\times$  over the fastest software implementation (dual-core with NEON acceleration), while also providing speedups of  $89.3\times$  and  $55.2\times$  over the single-core and dual-core scalar implementations respectively.

Table 4.8: Renderer throughput comparison.

| Configuration | Frame Rate (fps) | vs FPGA      |
|---------------|------------------|--------------|
| 1 core scalar | 0.672            | $89.3\times$ |
| 2 core scalar | 1.087            | $55.2\times$ |
| 2 core + NEON | 1.182            | $50.8\times$ |
| FPGA renderer | 60.0             | $1.0\times$  |

Table 4.9: FPGA renderer performance metrics.

| Metric                  | Value             |
|-------------------------|-------------------|
| Frame rate              | 60.0 fps          |
| Active pixel throughput | 18.43 Mpix/s      |
| Ray-marching throughput | 1,179.6 Msample/s |

## Chapter 5

# Interface and System Integration

### 5.1 System Overview

At the top level the programmable logic is one long pipeline (Fig. 5.1). The source generator feeds the FDTD solver, the solver’s output is reduced to a per-cell field magnitude, that magnitude is held in a double buffer, a bridge copies it into the renderer’s memory, and the renderer finally draws it out over HDMI. The guiding idea throughout is that each stage is loosely coupled to its neighbours: because the solver and the display naturally run at different and unrelated speeds, we never let one stage read another’s output until it is finished. The whole field datapath shares a single 25 MHz clock so that there are no clock-domain crossings on the path where data integrity matters most. Only the 3D renderer runs faster, at 100 MHz, and we bring its pixels back into the 25 MHz domain through an asynchronous FIFO.

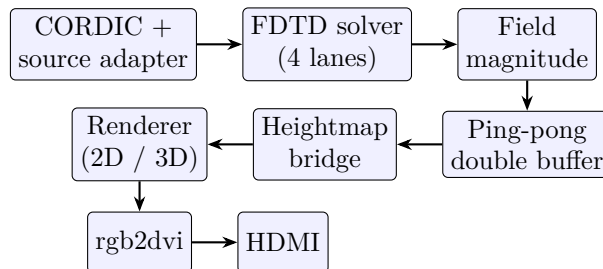


Figure 5.1: The end-to-end pipeline. Everything from the source generator to the bridge shares one 25 MHz clock. The renderer and HDMI serialiser follow on.

### 5.2 Building the Design from a Script

Rather than wiring the design together by hand in Vivado’s graphical block-design editor, we generate the whole thing from a Tcl script. This is because the block-design file Vivado produces is a large, auto-generated XML blob that cannot sensibly be reviewed or merged, whereas a script is readable, diffable, and regenerates an identical bitstream every time. We therefore commit the HDL and the script, which also makes it cheap to keep several design variants that differ by only a few lines.

The script runs in the phases of Fig. 5.2. Our own modules are first packaged as Vivado IP. The renderer is instead added as a module reference, which has the interesting quirk of requiring a Verilog (not SystemVerilog) top-level wrapper. One lesson worth noting is that we give every peripheral an explicit fixed address. We originally relied on Vivado’s automatic assignment, which orders peripherals alphabetically, and discovered that simply adding new blocks silently shifted all the existing ones and broke the addresses the notebook was written against.

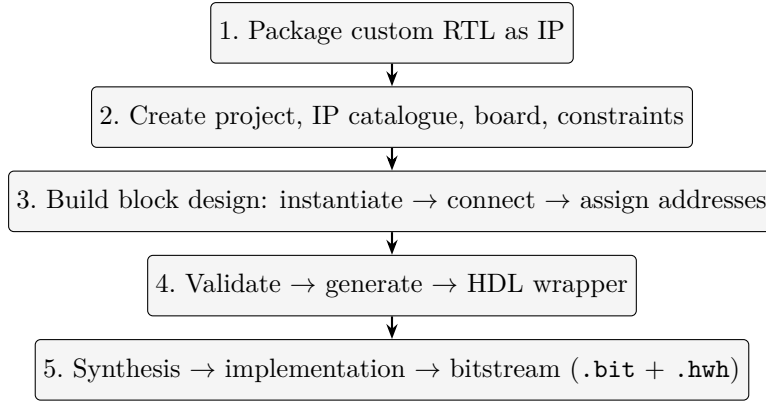


Figure 5.2: The scripted build. Phases 1–4 always run. Phase 5 is switched on by an environment variable, so the same script builds either just the project or the full bitstream.

### 5.3 Clocking

Every clock in the design comes from a single mixed-mode clock manager, the clocking wizard of Fig. 5.3, fed by the board’s 125 MHz reference (Table 5.1). Each domain has its own reset that is held until the wizard asserts its locked output, so no block starts running before its clock is stable. Where two domains have to meet we handle the crossing deliberately rather than hoping for the best. An asynchronous FIFO carries pixels from the 100 MHz renderer into the 25 MHz scan-out, a dual-clock block RAM holds the heightmap written on one clock and read on another, and the handful of control strobes that cross from the processor’s clock are passed through proper synchronisers.

| Clock   | Use                                       |
|---------|-------------------------------------------|
| 25 MHz  | field datapath, 2D renderer, video timing |
| 100 MHz | 3D ray-march renderer core                |
| 125 MHz | HDMI / TMDS serialiser                    |
| 50 MHz  | processor AXI register bus                |

Table 5.1: Clock domains derived from the 125 MHz board reference.

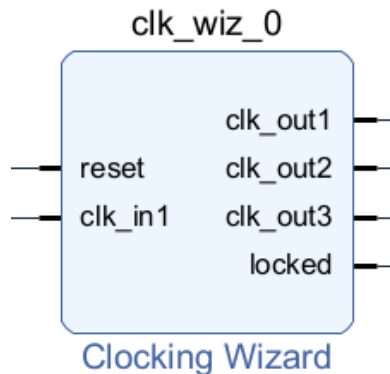


Figure 5.3: The clocking wizard: one 125 MHz input becomes the three on-chip clocks, with a locked output that holds the resets until they are stable.

## 5.4 Generating the Source

The wave is launched by a sinusoidal point source built from the two blocks in Fig. 5.4. A Xilinx CORDIC core does the trigonometry: it takes a phase angle on its input stream and returns the sine and cosine, and our `cordic_source_adapter` wraps it into something the rest of the system can use. The adapter accumulates the phase (advancing it one step per iteration, which is what sets the source frequency), drives the AXI-Stream handshake to and from the CORDIC, and scales the result by an amplitude the processor can change dynamically. It can also inject the difference of successive samples rather than the raw sine: this carries no average value, which matters because a moving source otherwise leaves a permanent “trail” of deposited energy behind it.

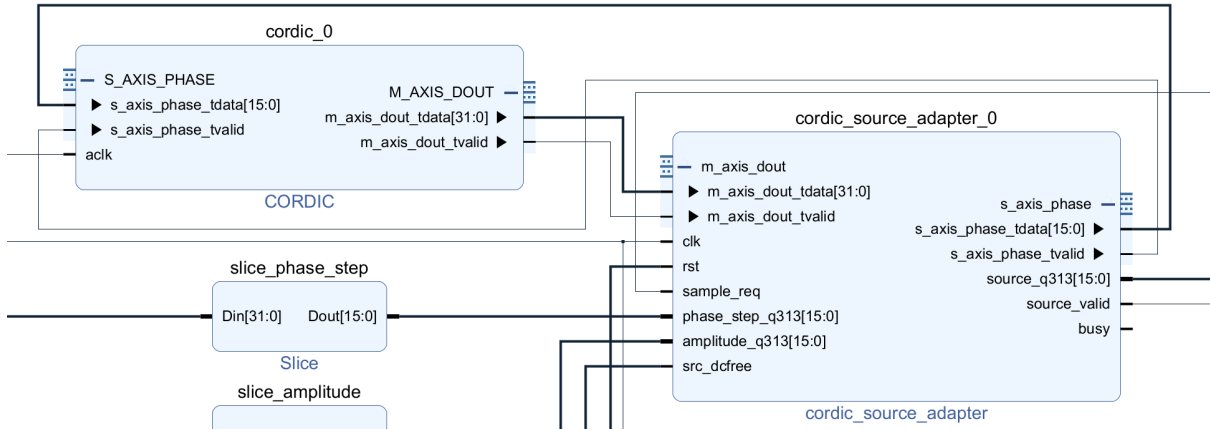


Figure 5.4: Block design of the source: the CORDIC core (left) feeds sine/cosine over an AXI-Stream link into `cordic_source_adapter`, which the slice blocks supply with the amplitude and phase-step set by the processor.

## 5.5 The Moving Source and the Doppler Effect

The system can also show the Doppler effect. If the source is made to travel through the grid, the wavefronts compress ahead of it and stretch out behind. Producing this cleanly is as much an architectural decision as a physical one, and it is a useful example of how work is divided between the processor and the fabric.

The effect depends entirely on the source moving smoothly. It advances only a small fraction of a cell on each FDTD iteration; the position is kept to a resolution of  $1/256$  of a cell, and it has to do so in lockstep with the solver, one small step for every iteration completed. This is precisely what rules out driving the motion from the processor. The PS reaches the fabric only over the AXI register bus through a Python loop, on a millisecond timescale, whereas the solver finishes an iteration in microseconds, resulting in a coarse and unsmooth implementation if motion were driven from the processor.

A small position generator inside the FDTD adapter keeps the source coordinate in a fixed-point accumulator, adds the velocity to it once per completed iteration, and reflects it off the PML walls so the source bounces around inside the grid rather than walking off the edge. Through the slice registers of Fig. 5.9, the processor sets the velocity, arms the motion and, because the real hardware runs far faster than is comfortable to watch, throttles the iteration rate so the sweep is slow enough to see. That division of labour, fast regular work in the fabric and slow high-level control from the processor, is the same pattern that recurs throughout the design.

## 5.6 Decoupling the Solver and the Display

This stage resolves a timing mismatch between the solver and the display. The solver finishes a new frame whenever it happens to, a rate that depends on the grid size and how much we have slowed it for viewing, however, the renderer must read a frame every sixtieth of a second to feed the display. If we simply pointed the renderer at the memory the solver was filling, it would now and then catch a frame half-written, with the top of the screen showing the new frame and the bottom the old one. This is the textbook producer-consumer problem, and we use the textbook answer: two identical memories and a small controller (Fig. 5.6). At any moment one memory is being written by the solver and the other is being read by the renderer, when the solver signals that a frame is complete, the controller swaps the two roles in a single clock cycle, as sketched in Fig. 5.5. Because the swap is atomic the renderer always sees a whole, self-consistent frame, and because each memory is dual-ported the reading and writing never collide. As both sides live in the same 25 MHz clock, none of this needs clock-crossing logic.

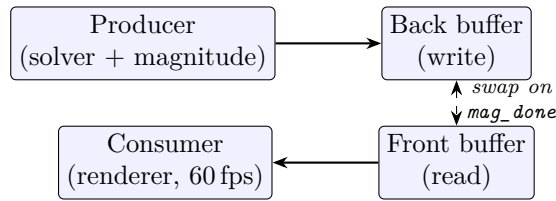


Figure 5.5: The double-buffering idea: the producer fills the back buffer while the consumer reads the front, and the two swap atomically each frame.

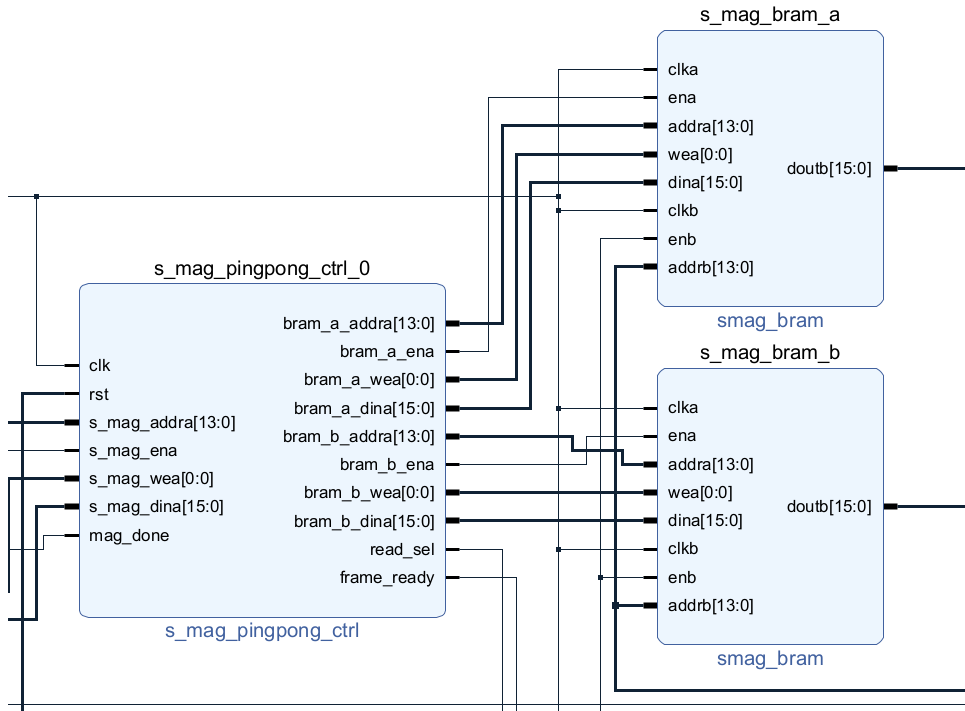


Figure 5.6: The same scheme in the block design: `s_mag_pingpong_ctrl` routes the solver's writes to whichever of the two `s_mag` memories is the current back buffer, and exports `read_sel/frame_ready` to the reader.

## 5.7 The Bridge to the Renderer

The renderer reads its terrain from its own private heightmap, so a bridge (Fig. 5.7) copies each finished frame across. It does this only during the display’s vertical blanking interval, when the renderer is issuing no reads, so the renderer can never catch the heightmap mid-update, the same tear-free principle as the double buffer, applied a second time.

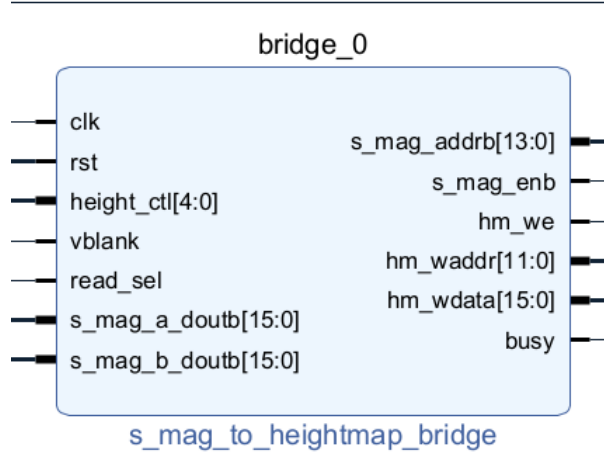


Figure 5.7: The `s_mag_to_heightmap_bridge`: it reads the front buffer (`read_sel`, `s_mag*_doutb`), waits for `vblank`, and writes the renderer heightmap (`hm_we/waddr/wdata`).

## 5.8 Driving It from the Processor

Everything the operator touches goes through the processor’s AXI bus (Fig. 5.8). The PS7 processing system drives an interconnect that fans out to a set of simple GPIO register blocks, alongside a reset controller that ties the whole AXI domain to the same locked clock. Because each GPIO register is 32 bits wide but usually carries several smaller values, slice blocks (Fig. 5.9) split each register into the individual fields the hardware consumes, and concatenation blocks gather status bits back into the read-back word. The resulting map is given in Table 5.2. The operator changes a value in the notebook and the simulation responds immediately, with no recompilation required.

| Address               | Function                                                    |
|-----------------------|-------------------------------------------------------------|
| 0x41200000            | control: amplitude, frequency, source position, mode, clear |
| 0x41210000            | motion: source velocity, speed throttle, camera load        |
| 0x41220000            | status (read-only): checksum and flags                      |
| 0x41230000–0x41250000 | camera basis (3D build)                                     |

Table 5.2: The AXI-GPIO register map exposed to the notebook.

## 5.9 Multi-Lane Parallelism

To speed the solver up we split the grid into four horizontal lanes and update them in lockstep, each with its own copy of the field memories. One iteration then takes 2052 cycles instead of 8196, four times faster, for an identical result. The cost falls on memory rather than logic: each lane needs its own memories and read ports, and neighbouring lanes exchange a single boundary row. Notably the four-lane version used fewer block RAMs than the single-lane one (108 against

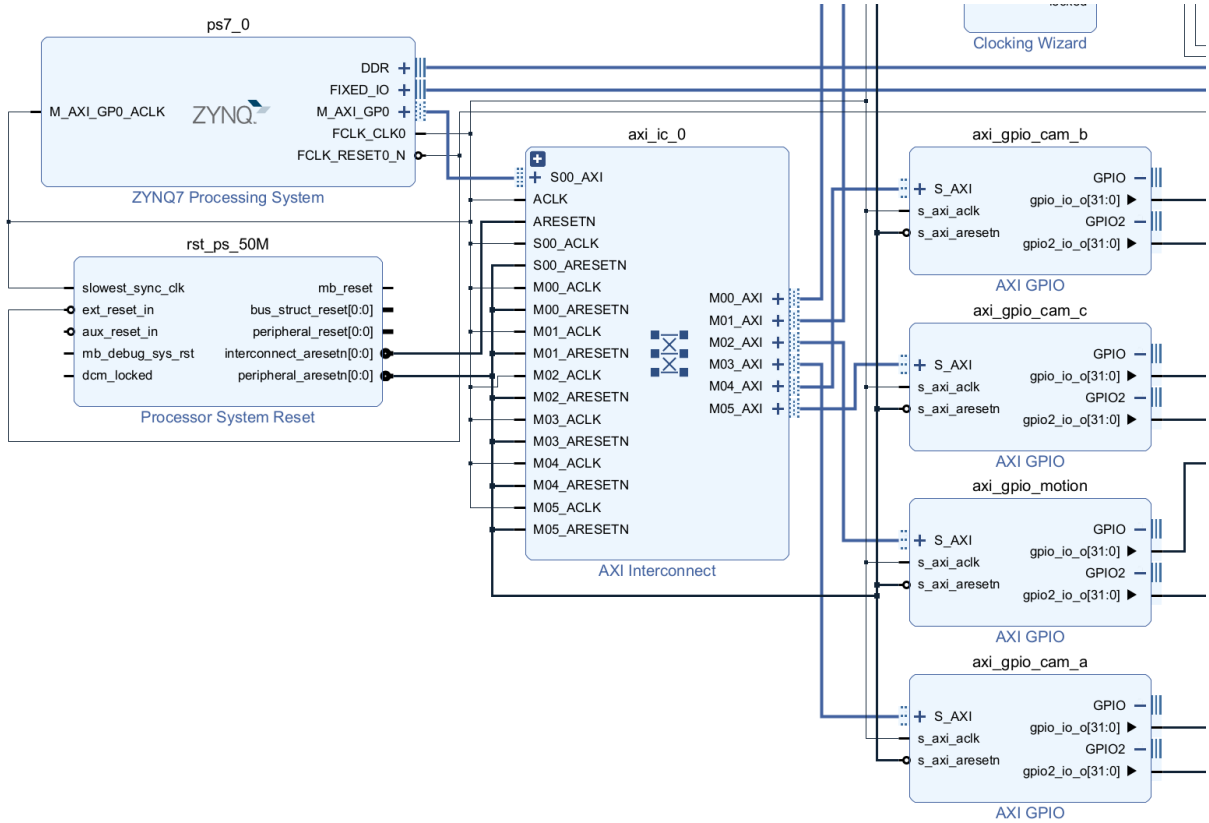


Figure 5.8: The processor (ZYNQ PS7) and its reset controller driving the AXI interconnect, which fans out to the GPIO register blocks (control, motion, status, and the camera channels).

114), because four smaller memories pack into the device’s RAM blocks more tightly than one large one. We implemented up to sixteen lanes (Section 3.11), but the integrated demonstrator runs four, as the higher throughput caused the wave to propagate too fast for our demonstration purposes.

## 5.10 The Two Demonstrations

The same solver drives two different renderers, and the choice between them is primarily a trade between block-RAM use and clock speed (Table 5.3). The first is a 2D contour heatmap (Fig. 5.10). It needs only a single heightmap memory, which uses drastically less BRAM. With the source set moving, it clearly shows the wavefronts bunching ahead and stretching behind, the Doppler effect. The second renders the field as a shaded three-dimensional landscape by ray-marching (Fig. 5.11). This view is more detailed but more resource-intensive, it needs around fifty copies of the heightmap, so it runs at 100 MHz and folds its memory reads across several cycles to fit, and its camera can be orbited live from the notebook. The 2D view is the clearer one for reading the physics, and the 3D view gives a more intuitive sense of the field’s shape in 3D reality.

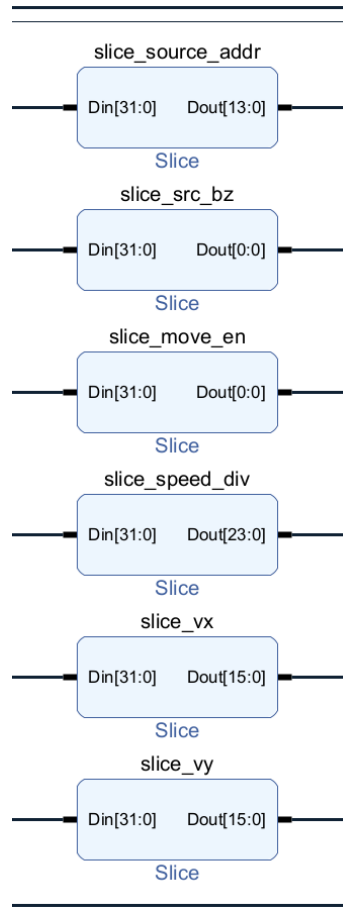


Figure 5.9: Slice blocks unpacking a single 32-bit GPIO register into separate control fields, here the source address, source-field select, and the moving -source velocity and throttle.

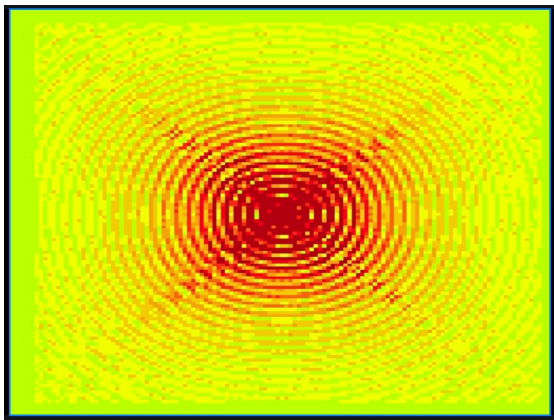


Figure 5.10: 2D contour heatmap: concentric wavefronts radiating from the point source.

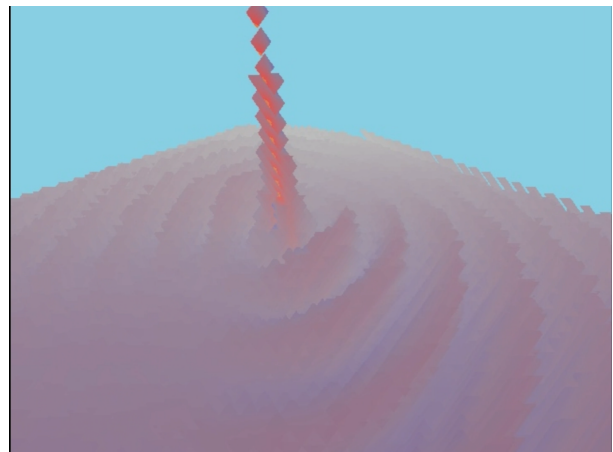


Figure 5.11: 3D ray-marched view: the field as a shaded heightmap, with the source rising at the centre.



|                 | 2D contour              | 3D ray-march                  |
|-----------------|-------------------------|-------------------------------|
| Heightmap BRAMs | 1                       | ~50                           |
| Clock           | 25 MHz                  | 100 MHz (1 px / 4 cyc)        |
| Strength        | clarity, finer grid     | visual impact, movable camera |
| Interaction     | moving source (Doppler) | orbiting camera               |

Table 5.3: The two demonstrations and their block-RAM versus clock trade-off.

## Chapter 6

# User Interface, Hardware and Demonstrations

### 6.1 What the demonstrator does, and the physics behind it

Electromagnetic fields are invisible and abstract, and the equations rarely make them intuitive. The demonstrator makes them tangible, letting a person interact with the simulated field and watch it respond in real time (Table 6.1).

| Property      | Interactive Waves                |
|---------------|----------------------------------|
| Physics shown | Time-dependent wave (Maxwell)    |
| Input device  | Touching the touch panel         |
| Panel reports | $XY$ position                    |
| Live knob     | Conductivity sets the wave speed |

Table 6.1: The interactive wave demonstrator.

The key hardware requirement is an accurate (x,y) position from the touch surface. This lets the user place sources or walls directly on the simulation grid. The microcontroller gathers every control via polling and sends a fixed message by wire to the FPGA board, which forwards it to the PL (Figure 6.1).

### 6.2 The user interface

All controls sit on one front face (Table 6.2): one touch surface, four switches and seven rotary knobs.

### 6.3 Power supply and analog conditioning

The whole subsystem runs from the single 5 V USB feed it shares with the FPGA board, which also ties their grounds together. The microcontroller board regulates this supply to 3.3 V for the controls. The rails are listed in Table 6.3.

The seven knobs are referenced directly to the board's regulated **3.3 V (3V3) pin**: each is a potentiometer between 3V3 and ground feeding an analog input. The switches simply pull their pin to ground, held high otherwise by internal pull-ups.

The analog controls require no additional conditioning stage: each potentiometer is connected directly between 3.3 V and ground, with its wiper feeding an ESP32 analog input (Figure 6.3).

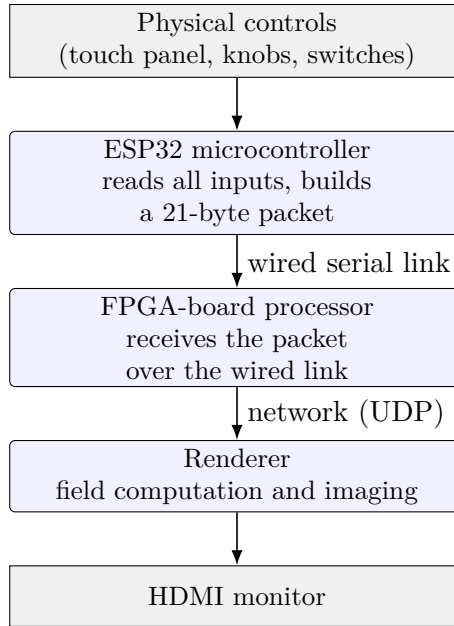


Figure 6.1: Data path of the demonstrator within the wider system. This chapter covers the top two blocks. Everything below the serial link belongs to the other teams.

| Control                   | Type             | Function                                                               |
|---------------------------|------------------|------------------------------------------------------------------------|
| Touch panel               | 4-wire resistive | $XY$ position                                                          |
| 2D/3D                     | switch           | flat or rendered-surface display                                       |
| Wall/Source               | switch           | what a press places                                                    |
| Clear                     | switch           | wipe the scene; set at power-up to recalibrate                         |
| DC/AC                     | switch           | toggles the source between a DC steady field and an AC sinusoidal wave |
| Amplitude                 | knob             | source amplitude                                                       |
| Conductivity              | knob             | wave speed                                                             |
| Field type                | knob (3 zones)   | show $E$ , $B$ or Poynting                                             |
| Yaw, Pitch, Zoom, Z-scale | 4 knobs          | 3D camera view                                                         |

Table 6.2: The front-panel controls.

| Rail   | Where it comes from                    | What it feeds                                          |
|--------|----------------------------------------|--------------------------------------------------------|
| +5 V   | USB supply (shared with the FPGA)      | microcontroller board and shared system supply         |
| 3.3 V  | the microcontroller's on-board 3V3 pin | the top end of all seven knobs                         |
| Ground | common to everything                   | microcontroller, touch panel, controls, and FPGA board |

Table 6.3: The supply rails of the control subsystem.

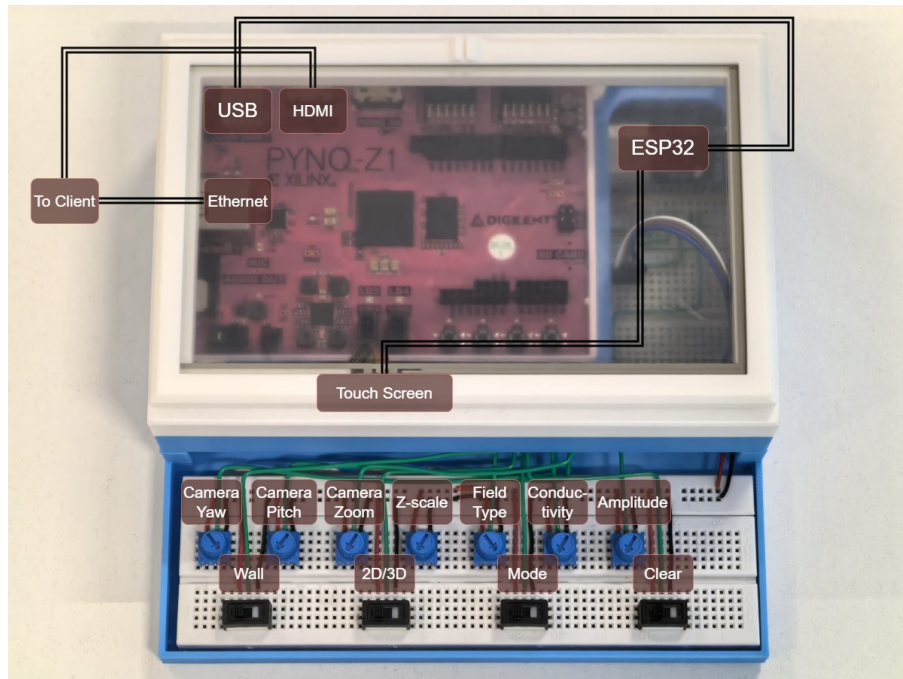


Figure 6.2: The front-panel control layout.

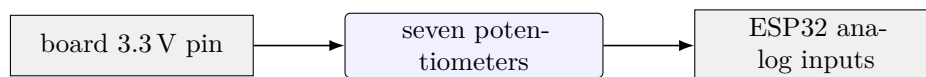


Figure 6.3: Analog-control path: the seven potentiometers are referenced directly to the board's 3.3 V pin and read by the ESP32 analog inputs.

Each sub-circuit was verified in SPICE before assembly.

## 6.4 The microcontroller and the control link

The subsystem is built around an ESP32 with a multi-channel analog-to-digital converter, ample I/O, a hardware serial port and Arduino tooling. The panel is driven with plain digital levels, and calibration absorbs the small offset.

Eighteen of the roughly two dozen usable pins are occupied (Table 6.4). This effectively fills the chip: a couple of controls sit on an input-only pin and on two start-up “strapping” pins.

| Pins                | Purpose                                                                                |
|---------------------|----------------------------------------------------------------------------------------|
| six pins            | touch panel — two axis-drive pairs plus two sense lines (Section 6.5)                  |
| seven analog inputs | the seven knobs: amplitude, conductivity, field-type, yaw, pitch, zoom, vertical scale |
| four digital inputs | 2D/3D, wall/source, clear, and DC/AC switches                                          |
| two serial pins     | transmit and receive for the link to the FPGA board                                    |

Table 6.4: How the occupied pins are allocated.

The firmware repeats a fixed cycle about fifty times a second (Figure 6.4), packing everything into a fixed control packet with a start marker and a checksum (Table 6.5). When no touch is valid the position bytes are zero and a flag tells the receiver to place nothing that frame. The ESP32 cannot send UDP itself (its radio is off), so the packet goes by wire to the FPGA board’s processor, which forwards it to the renderer (Figure 6.1).

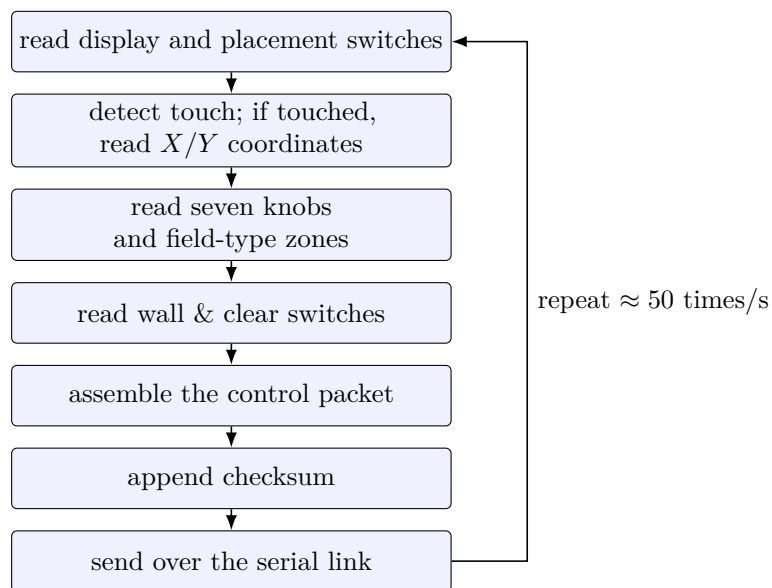


Figure 6.4: The fixed firmware cycle, repeated about fifty times per second.

## 6.5 The touch panel: sensing and calibration

Getting clean, trustworthy coordinates from the panel took more effort than any other part. The panel is two resistive layers held a hair apart. A press shorts them at the contact point. Position is read by driving one layer as a voltage ramp and sensing the contact voltage on the floating layer, swapping the layers for the two axes, three short phases per loop (Table 6.6).

| Bytes         | Contents                                                                   |
|---------------|----------------------------------------------------------------------------|
| 1             | start marker (a fixed recognisable byte)                                   |
| 1             | flags: 2D/3D, field type, wall-or-source, clear active, DC/AC, touch valid |
| 2 + 2         | touch position, as a column and row on the display grid                    |
| 2             | source amplitude                                                           |
| 2             | conductivity (wave speed)                                                  |
| 2 + 2 + 2 + 2 | the four camera-view knobs: yaw, pitch, zoom, vertical scale               |
| 1             | checksum over the message body                                             |

Table 6.5: The fixed control packet.

| Phase        | Layer driven              | Sensed on      | Result     |
|--------------|---------------------------|----------------|------------|
| Touch detect | Y- pulled low             | X+ via pull-up | pressed?   |
| Read X       | X layer (X+ high, X- low) | a Y terminal   | X position |
| Read Y       | Y layer (Y+ high, Y- low) | an X terminal  | Y position |

Table 6.6: The three read phases of the four-wire panel.

Each axis is sensed on a dedicated low-leakage input, so the high-impedance contact node stays linear, and read with the converter’s millivolt calibration. Readings are filtered: a median for touch detection and a trimmed mean of a dozen samples per coordinate, taken after a short settling pause.

The corners never read the true extremes, so the user sets the range with a quick two-corner press, one corner, then the opposite, and only firm presses are learned. Each press is then converted as in Figure 6.5, and the finished grid cell goes straight into the packet.

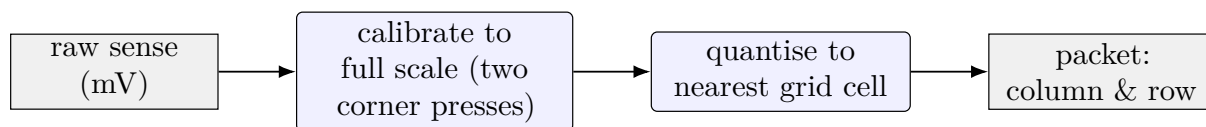


Figure 6.5: Touch-coordinate pipeline: each raw reading is calibrated, quantised to a display-grid cell, and sent in the packet.

## 6.6 The web interface

The physical console described above gives a tactile route into the simulation; a browser-based interface gives a second, equally complete route, and adds the on-screen scaffolding with live equations, labelled colour scales and guided pages that turns a moving picture into a teaching instrument. It runs on any laptop on the same network, so a whole class can follow and drive the field without crowding the hardware.

### 6.6.1 Requirements

The interface was designed against the prioritised requirements in Table 6.7, which place its educational role first.

| Requirement     | What it means here                                                                                                                 |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Educational     | Each control sits beside the equation and field quantity it affects, so a student sees parameter, mathematics and result together. |
| Functionality   | Every simulation parameter is adjustable live, and the user can switch between the 2-D field map and the 3-D rendered surface.     |
| Accessibility   | The colour map is selectable for the common colour-vision deficiencies, and the tool runs on any laptop on the subnet.             |
| Interchangeable | The web controls and the physical console drive the same hardware and can be swapped without conflict.                             |
| Security        | The board accepts control messages only from within the local subnet; all parameters are range-clamped before reaching hardware.   |

Table 6.7: Prioritised requirements for the web interface.

### 6.6.2 Architecture: two data planes

The interface is built around two independent data planes (Figure 6.6). A data plane is the complete software path that one kind of data follows from end to end, its transport, the server endpoint that handles it, and any processing along the way. Our system has two such paths, and they share no code and compete for no common resource: one carries control settings to the board, the other carries the rendered picture back to the screen.

The control plane carries parameters. When the user moves a slider, the browser sends the new value over a persistent WebSocket to a lightweight server running on the FPGA board’s processor. The server then converts each message into the number format the hardware uses and writes it to the hardware registers, so the change shows up on the next frame.

The video plane carries the picture itself. One option would be to have the board’s processor capture each frame, compress it, and send it to the browser as an image, but that is slow. Instead, we take the FPGA’s HDMI output (the same signal that would go to a monitor) and feed it through a small USB capture card into the laptop, where the browser draws it straight onto the page. The picture therefore never passes through the board’s processor at all.

Keeping the two planes separate is the key design choice: control messages reach the board with low latency, while the rendered field returns at full frame rate, because neither path has to wait for the other.

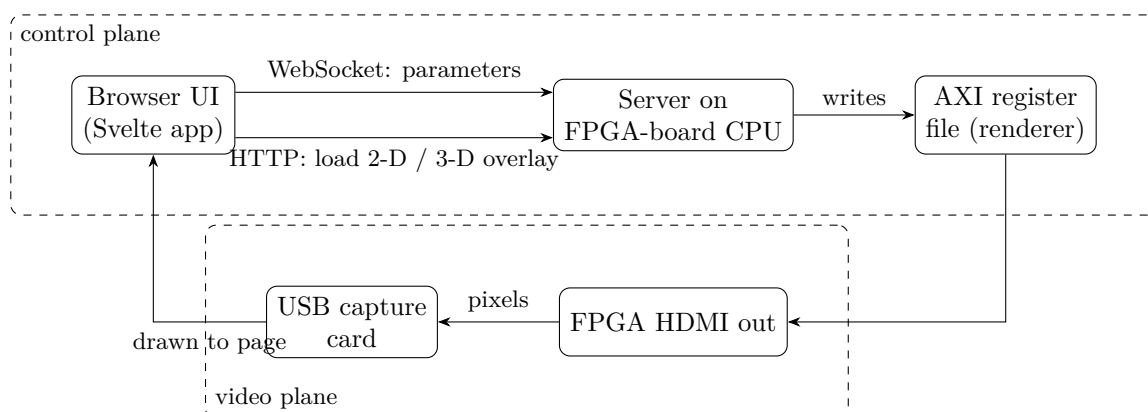
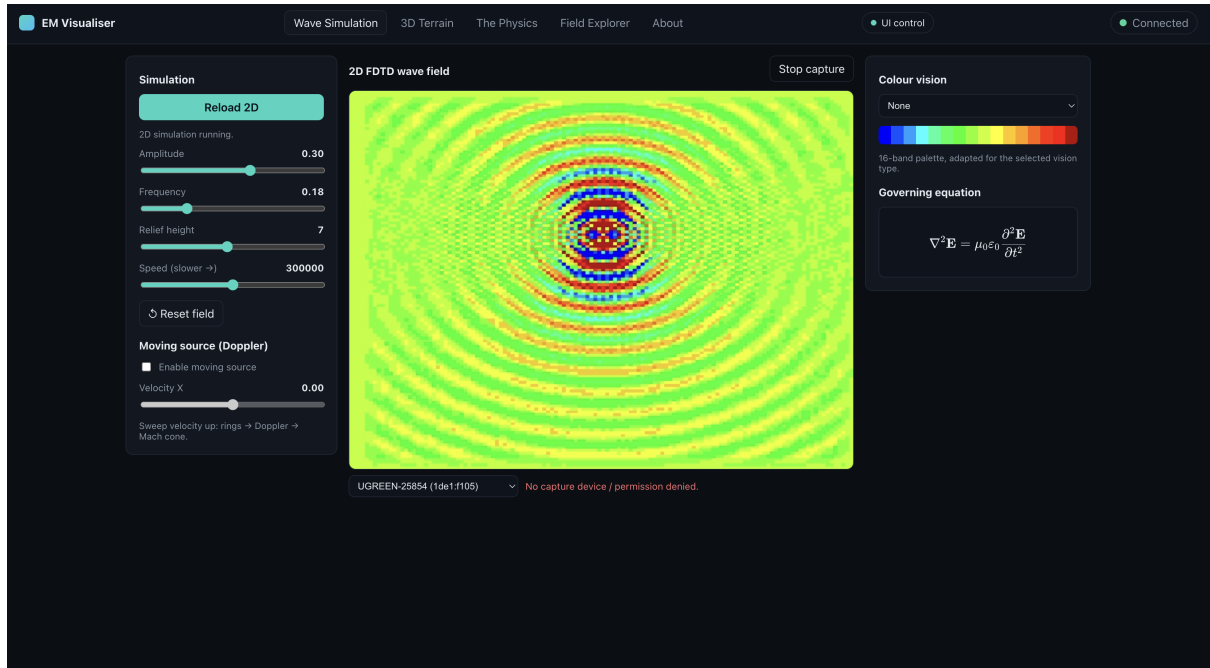


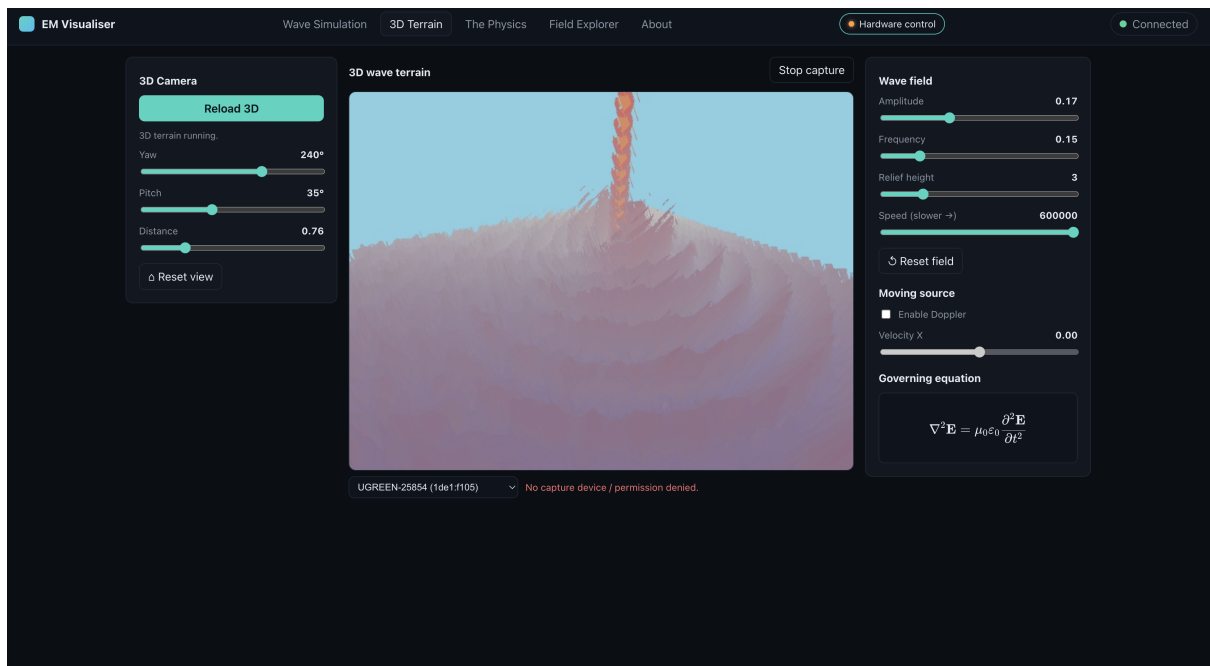
Figure 6.6: The two data planes. Control parameters travel over a WebSocket and are translated to register writes on the board; the rendered field returns as live video through a capture card, bypassing slow processor-side image encoding.

A teaching tool should let a user change what is computed, not only its parameters. A control

selects either the 2-D field map or the 3-D rendered surface; the server responds by loading the matching FPGA configuration (overlay) onto the board on demand.



(a) 2-D FDTD wave field: live simulation controls on the left, colour-vision palette and governing equation on the right.



(b) 3-D rendered terrain of the same field, with orbit camera controls (yaw, pitch, distance) and the shared wave parameters.

Figure 6.7: The browser interface reconfiguring the accelerator on demand.

### 6.6.3 One simulation, two control surfaces

The physical console and the web sliders write the same eight-register control file, so they must not drive the field simultaneously. A single flag on the server resolves this: a toggle switches



ownership between interface and hardware. In hardware mode the incoming web parameters are ignored and the on-screen sliders instead track the live knob values streamed back from the console, so the display always reflects what is physically set.

#### 6.6.4 Designed as a teaching instrument

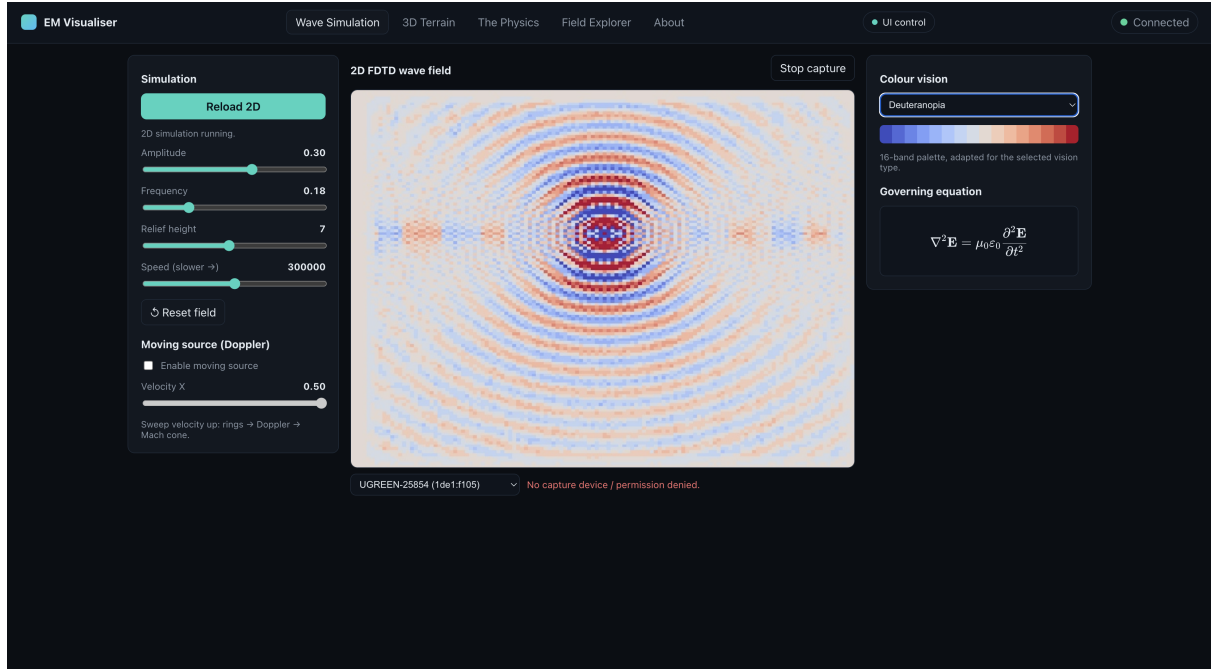
Several deliberate choices serve the educational goals our system sets out to meet.

| Choice                           | How it serves the educational requirement                                                                                                                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page-per-concept                 | The app is split into focused pages, each pairing its controls with the governing equation, typeset live, and a short explanation, so parameter, mathematics and resulting field are read together.                                             |
| Colour-vision-deficiency support | The colour map switches to palettes safe for protanopia, deuteranopia and tritanopia. Only the 16-entry palette is recoloured, not every pixel, so the change and its legend update instantly and the field stays unambiguous for every viewer. |

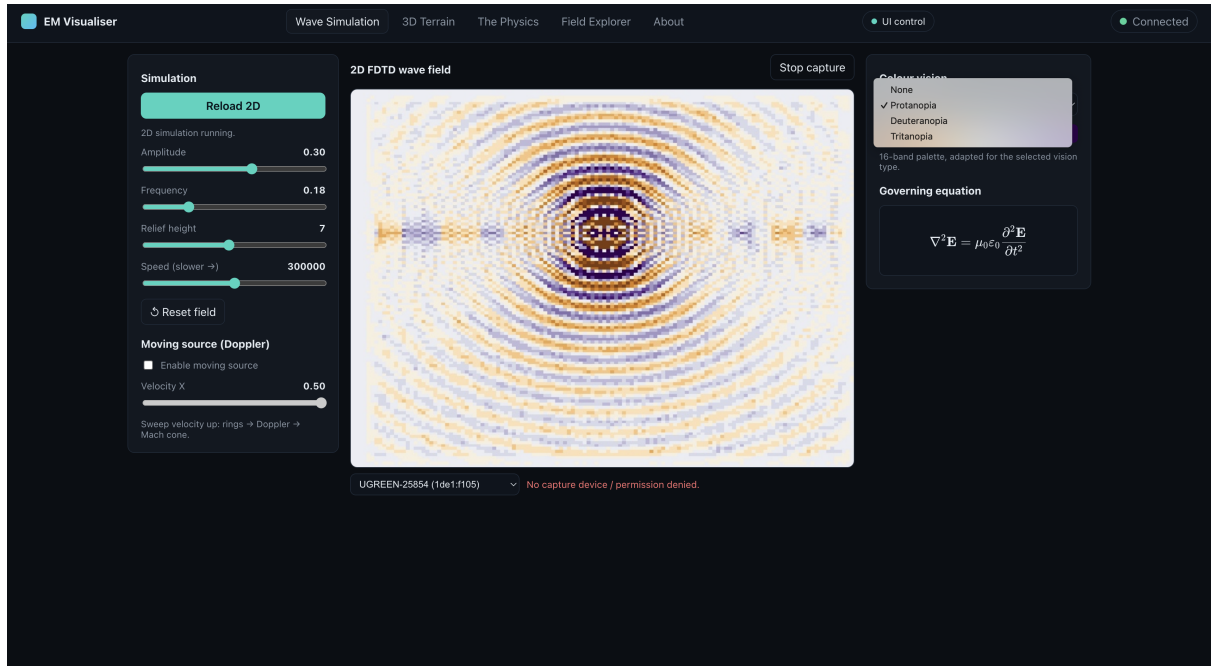
Table 6.8: Educational design choices in the web interface.

| Page            | Role                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------|
| Wave Simulation | 2-D field cockpit: source amplitude, frequency, wave speed, reset, moving-source (Doppler), and colour-map controls. |
| 3-D Terrain     | The same field as an orbitable rendered surface, with camera (yaw, pitch, distance) and the same wave controls.      |
| The Physics     | The governing equations and the meaning of each displayed field quantity.                                            |
| Field Explorer  | Guided parameter presets that reproduce the key phenomena.                                                           |

Table 6.9: The page structure of the web interface.



(a) Deuteranopia palette: the diverging blue–white–red map keeps crest and trough distinguishable without relying on the red–green axis.



(b) Protanopia palette: the same field re-mapped to an orange–white–blue scale.

Figure 6.8: Colour-vision-deficiency support. Selecting a deficiency type recolours only the 16-entry palette and its legend, so the field reads unambiguously for every viewer; the underlying simulation is unchanged.

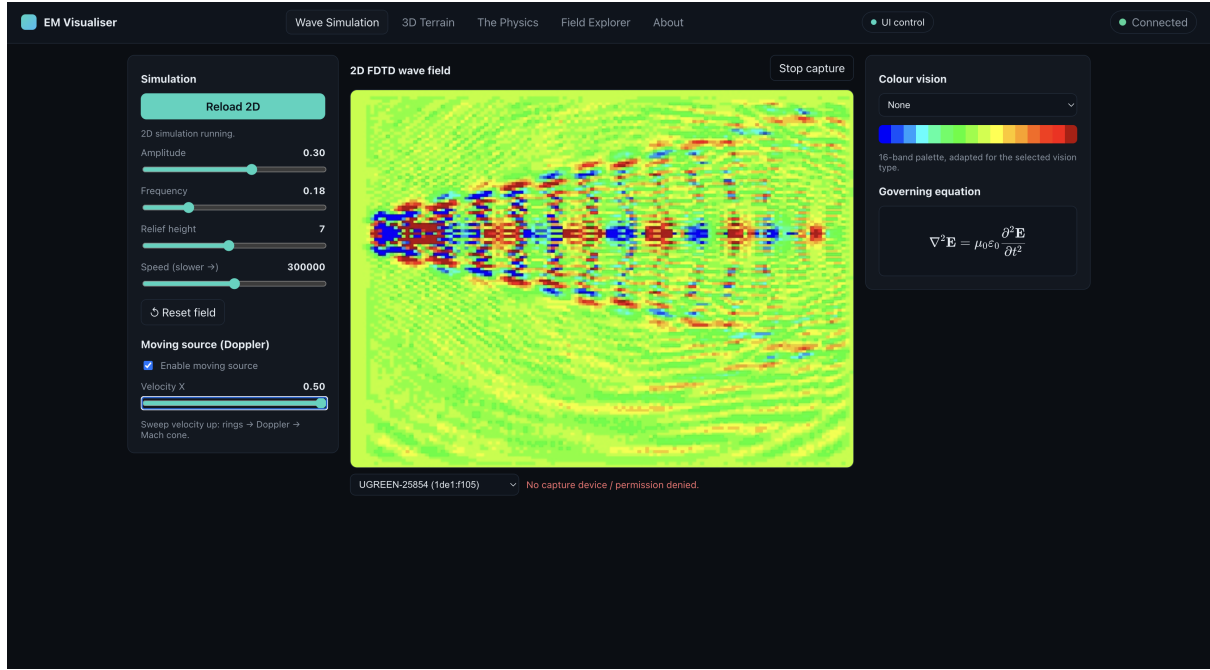


Figure 6.9: Enabling the moving source: with a non-zero source velocity the concentric rings compress ahead of the source and stretch behind it, reproducing the Doppler effect.

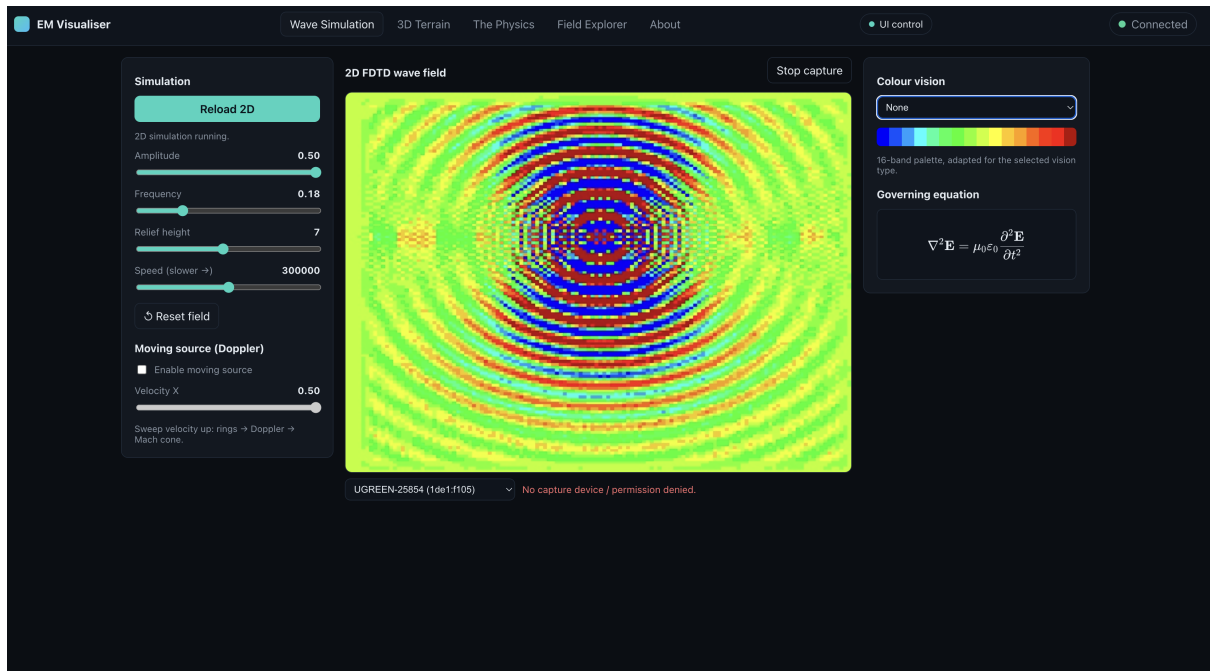


Figure 6.10: Driving the amplitude to its maximum: the rendered field responds in the next frame, demonstrating that every parameter is adjustable live with no recompilation or reload.

## 6.7 Enclosure and mechanical design (box modelling)

A showcase demonstrator needs a real housing, so a custom enclosure was modelled and 3D-printed. It is a single open-bottomed cover, an upturned tray, that drops over the baseboard-mounted electronics. No internal fixings, a clean screwless top face, the FPGA left open in its bay. Its key features are listed in Table 6.10 and its top-face layout in Figure 6.11.

| Feature                  | Size / detail                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Overall cover<br>(W×D×H) | 179 × 178 × 67 mm                                                                                                        |
| Walls / top plate        | 3 mm / 5 mm                                                                                                              |
| Touch-panel window       | hand-shaped recess, on the back edge of lid for setting input wire                                                       |
| FPGA bay                 | 127 × 92 mm open cut-out, back-left; only HDMI, USB and power supply remain exposed, while the board buttons are covered |
| Control strip            | open breadboard strip exposing all controls directly: 7 knobs and 4 switches                                             |
| Companion parts          | panel frame (bezel) + paper frame (holds the conducting sheet flat)                                                      |

Table 6.10: Key features of the 3D-printed cover.

The panel drops into a shallow stepped recess and rests on a thin internal lip, leaving its active area exposed through a window while its bezel sits flush with the top. A slim panel frame is used to clamp the touch panel. (Figure 6.12).

At 179 × 178 mm the cover fits on a standard print bed in a single piece. The finished enclosure is the cover plus the panel frame and paper frame. Breadboards mount to the baseboard, the framed panel and the knobs and switches fit the top face, and the cover drops over with cables exiting the rear slot.

## 6.8 Interactive Wave Simulation

This solves the full time-varying Maxwell equations on the FPGA using FDTD and renders three field quantities, the electric field magnitude  $|\mathbf{E}|(x, y, t)$  and the Poynting vector magnitude  $|\mathbf{S}|(x, y, t)$ . The user places wave sources via the resistive touch panel and can change material properties and add reflecting boundaries.

### 6.8.1 Panel 1: Electric field $|\mathbf{E}|(x, y, t)$

Panel 1 shows the scalar magnitude  $|\mathbf{E}| = \sqrt{E_x^2 + E_y^2}$ . The display shows expanding circular rings centred on the source position.

With two sources, the fields superimpose.

In 3D mode,  $|\mathbf{E}|$  is mapped to height. Wavefronts appear as sharp propagating ridges.

### 6.8.2 Panel 2: Magnetic field $B_z(x, y, t)$

Panel 2 shows  $B_z(x, y, t)$  with the same bipolar colour map as Panel 1. It also shows expanding rings, but they are spatially and temporally shifted relative to those in Panel 1.

The Faraday equation couples  $\mathbf{E}$  and  $\mathbf{B}$ :  $\partial B_z / \partial t \propto \nabla \times \mathbf{E}$ . So  $B_z$  is proportional to the time integral of the curl of  $\mathbf{E}$ . Integrating a sinusoidal function shifts the phase by 90°:

$$E(t) = E_0 \sin(\omega t) \Rightarrow B_z(t) \propto \int E dt \propto -\cos(\omega t) = \sin\left(\omega t - \frac{\pi}{2}\right) \quad (6.1)$$

This is visible on the display, as when the electric field rings are at maximum,  $B_z$  is minimum, and vice versa. The two fields take turns, alternating stored energy at twice the source frequency. This alternation is the mechanism sustaining wave propagation. The changing  $\mathbf{E}$  continuously generates  $\mathbf{B}$  via Ampère–Maxwell (3.2), and the changing  $\mathbf{B}$  continuously regenerates  $\mathbf{E}$  via Faraday. Neither field alone is sufficient; the wave requires both to self-sustain.

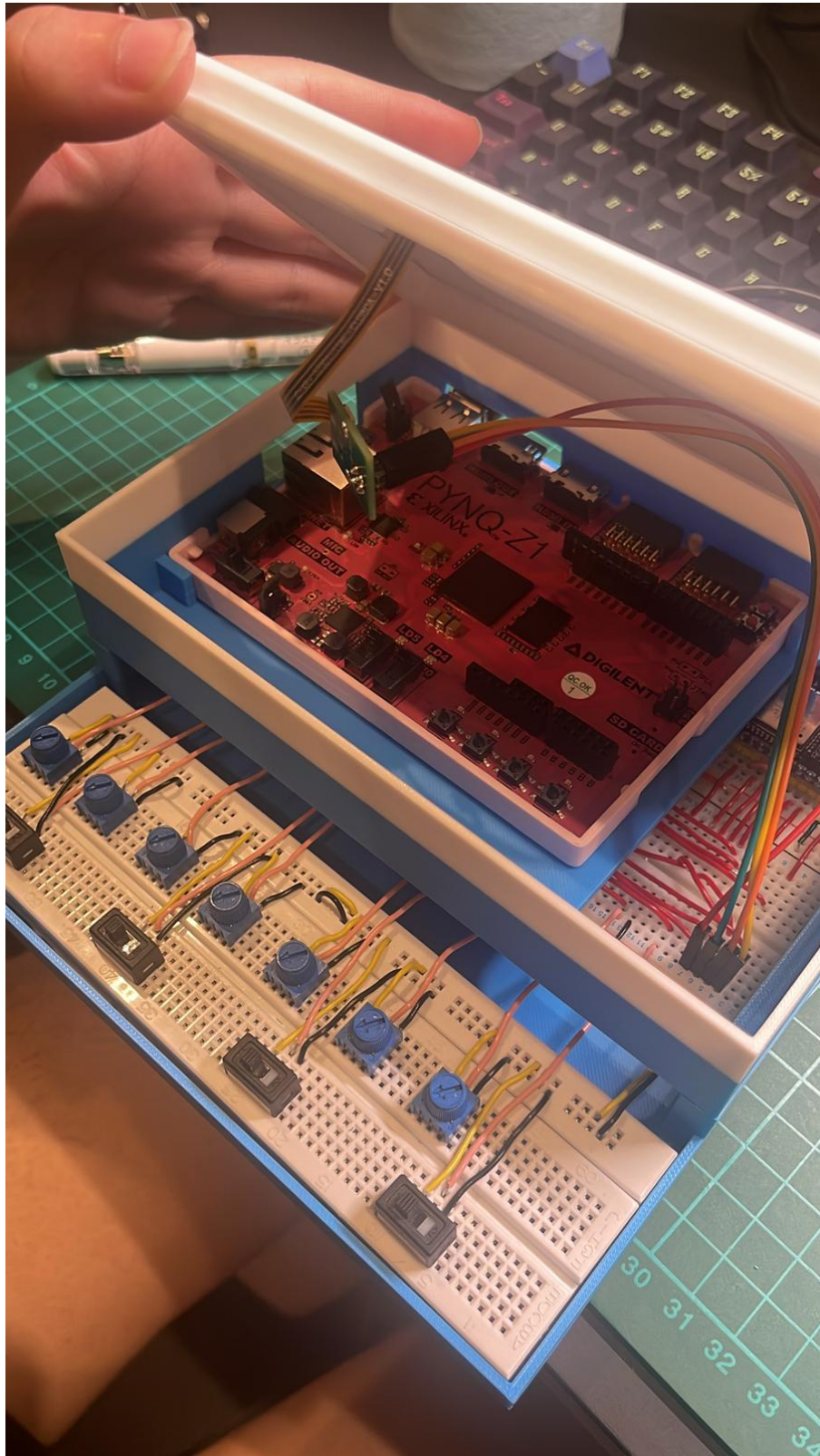


Figure 6.11: Top-view layout of the cover.



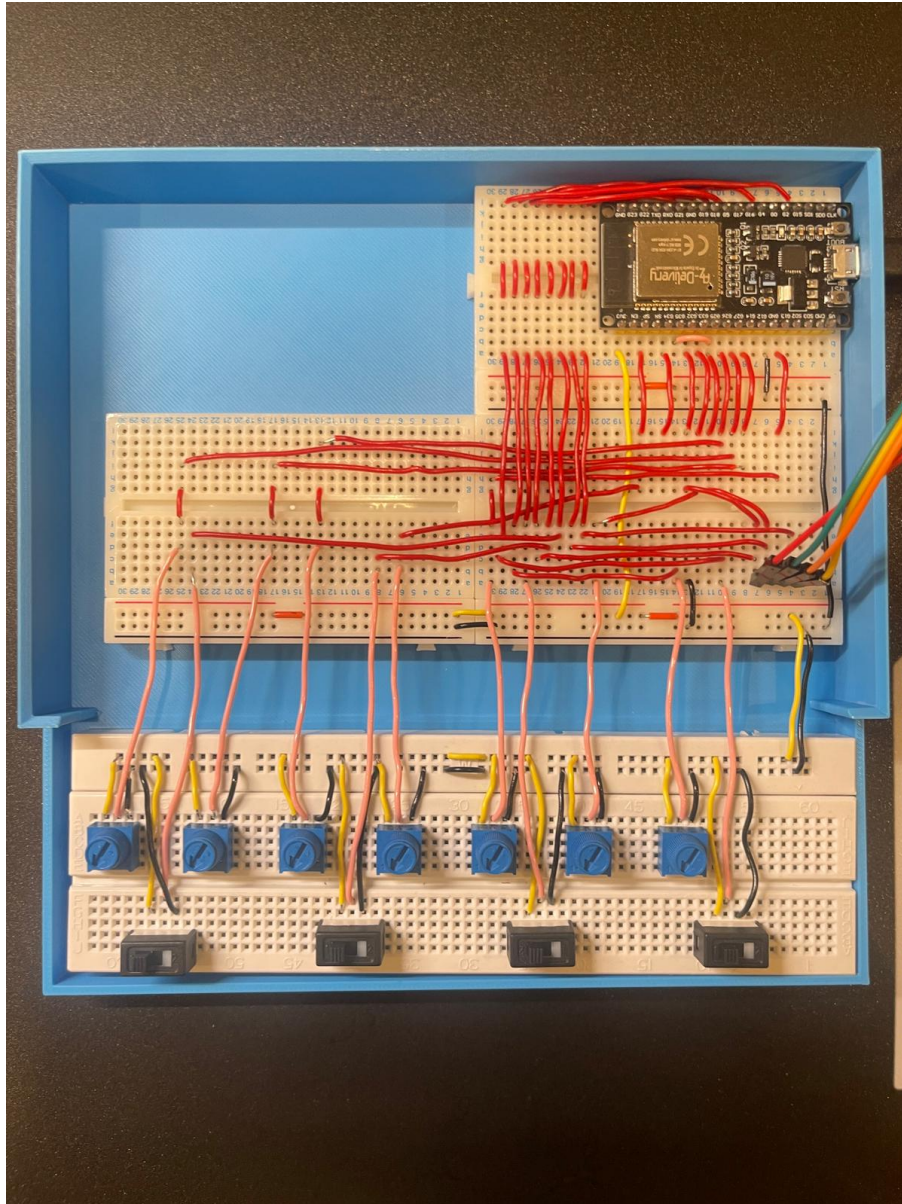


Figure 6.12: The printed touch-panel frame.

### 6.8.3 Panel 3: Poynting vector $|\mathbf{S}|(x, y, t)$

Panel 3 shows  $|\mathbf{S}|$  computed from the FDTD arrays using

$$\mathbf{S} = \frac{\mathbf{E} \times \mathbf{B}}{\mu_0} \quad (6.2)$$

$|\mathbf{S}| \geq 0$  always. From a single source the display shows a bright central region fading radially outward. With two sources it shows bright fringes where energy concentrates and exactly black nulls where it does not.

Both  $\mathbf{E}$  and  $\mathbf{B}$  are zero at destructive interference nulls. Therefore  $|\mathbf{S}| = 0$  exactly at those positions.

The time-averaged Poynting vector is:

$$\langle \mathbf{S} \rangle = \frac{1}{2\mu_0} \text{Re}(\mathbf{E} \times \mathbf{B}^*) \quad (6.3)$$

In the far field,  $\langle \mathbf{S} \rangle$  points radially outward, meaning energy radiates away permanently. Near the source ( $r \lesssim \lambda/2\pi$ ), the instantaneous Poynting vector oscillates back and forth without escaping; this is the reactive near field.

### 6.8.4 Relationship between the three panels

The three panels together give a complete physical picture:

- Panel 1 ( $|\mathbf{E}|$ ): where the electric force on charges is strong.
- Panel 2 ( $B_z$ ): the magnetic field,  $90^\circ$  out of phase with Panel 1, generated by the changing electric field via Faraday's law.
- Panel 3 ( $|\mathbf{S}|$ ): where electromagnetic energy flows and at what rate. Together these three panels give a complete picture of the field.

### 6.8.5 More demonstrations

#### Reflection and diffraction

Setting  $E_z = 0$  at a row of grid cells imposes a perfect electric conductor boundary condition, the tangential electric field must vanish at a conductor surface. Incident waves reflect with inverted phase, and so the superposition of incident and reflected waves forms a standing wave in front of the wall. A gap allows transmission, with waves spreading out in a semicircle on the far side. The angular spread scales as  $\lambda/d$ , where  $d$  is the gap width.

#### Refraction at a dielectric interface

Different  $\varepsilon_r$  values in the left and right halves of the grid give different wave speeds via (3.5). Wavefronts must remain continuous across the boundary, so they bend: Snell's law  $n_1 \sin \theta_1 = n_2 \sin \theta_2$  (with  $n = \sqrt{\varepsilon_r \mu_r}$ ) describes the bending angle. Adjusting the  $\varepsilon_r$  slider changes the refractive index ratio and the bend angle updates in real time.

#### Energy density

The electromagnetic energy density:

$$u = \frac{\varepsilon_0}{2} (E_x^2 + E_y^2) + \frac{B_z^2}{2\mu_0} \quad (6.4)$$

is always non-negative. This simplifies to  $u = E_x^2 + E_y^2 + B_z^2$ . At destructive interference nulls both  $\mathbf{E}$  and  $\mathbf{B}$  vanish, so  $u = 0$  exactly. The energy density display complements Panel 3 (Poynting vector)  $|\mathbf{S}|$ , which shows where energy flows, while  $u$  shows where energy is stored.

## Chapter 7

# Evaluation and Conclusion

### 7.1 Integrated system performance

As shown in Tables 7.1 and 7.2, the complete FPGA pipeline achieves a sustained throughput of 60 frames per second while performing both electromagnetic field simulation and volume rendering. The solver itself completes 48,828 iterations per second, far more than the display requires, so the frame rate is set by the 60 Hz HDMI interface rather than by computation. At this operating point the design processes 18.43 Mpix/s and performs approximately 1,179.6 Msample/s of ray-marching computations. Compared with the fastest software implementation on the Cortex-A9 (dual-core with NEON acceleration), the FPGA implementation achieves a speedup of  $76.1\times$ , while also providing speedups of  $124.5\times$  and  $81.1\times$  over the single-core and dual-core scalar CPU implementations respectively.

Table 7.1: End-to-end pipeline throughput comparison.

| Configuration | Pipeline Rate (fps) | vs FPGA       |
|---------------|---------------------|---------------|
| 1 core scalar | 0.482               | $124.5\times$ |
| 2 core scalar | 0.740               | $81.1\times$  |
| 2 core + NEON | 0.788               | $76.1\times$  |
| FPGA pipeline | 60.0                | $1.0\times$   |

Table 7.2: FPGA pipeline performance metrics.

| Metric                  | Value             |
|-------------------------|-------------------|
| Solver iteration rate   | 48,828.1 /s       |
| HDMI-limited rate       | 60.0 fps          |
| Pipeline rate used      | 60.0 fps          |
| Active pixel throughput | 18.43 Mpix/s      |
| Ray-marching throughput | 1,179.6 Msample/s |



## 7.2 Initial vs. final design

| Subsystem              | Initial expectation                                                                  | Final result                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Concept                | Conducting paper with electrodes and probe; separate non-interactive simulation mode | Fully interactive 3D dynamic EM simulation; touch-panel wave injection with knobs and switches for angle and medium |
| Solver grid            | $192 \times 192$                                                                     | $128 \times 128$                                                                                                    |
| Solver lanes           | 4-lane architecture                                                                  | 16-lane peak (4-lane in integrated demo)                                                                            |
| Boundaries             | Zeroed boundaries                                                                    | UPML                                                                                                                |
| Solver passes          | 3-pass                                                                               | 2-pass                                                                                                              |
| Sources / walls        | Multiple sources and walls                                                           | Multiple sources and walls                                                                                          |
| Wave effects           | Refraction, diffraction, reflection                                                  | Refraction, diffraction, reflection                                                                                 |
| Field outputs          | $ E $ and $ S $                                                                      | $ E $ , $ S $ , $E_y$ and $B_z$                                                                                     |
| Renderer interpolation | Dynamic shading                                                                      | Bilinear-interpolated shading and silhouette                                                                        |
| Renderer coverage      | $\geq 90\%$ hit coverage                                                             | 97% hit coverage                                                                                                    |
| Renderer type          | Ray-marching renderer                                                                | Ray-marching renderer                                                                                               |

Table 7.3: Initial design expectations against the delivered system. The final design exceeds the original proposal on interactivity, lane parallelism, boundary modelling, render depth, and interpolation quality.

## 7.3 Limitations and future work

The current FDTD engine has several limitations. Because the solver produces new states faster than the renderer can consume them, the ping-pong buffer drops intermediate frames and only ever displays the newest. The Q3.13 fixed-point format saturates on strong fields and loses precision on weak ones, limiting dynamic range, while the  $128^2$  grid cannot resolve fine wave detail. On the hardware side, the 16-lane solver moves the wave too quickly to watch comfortably in a demo, so the integrated system is limited to four lanes. The primary limitations of the renderer come from the use of height-field ray marching. Because the geometry is represented as a single height value at each  $xy$  location, structures containing overhangs cannot be represented and surfaces viewed from below cannot be rendered. Furthermore, the discrete ray-marching step size introduces a sampling limitation: sharp peaks whose width is smaller than the marching interval may not be detected reliably, causing them to appear clipped in the rendered output.

Given more time, for the FDTD solver we would try to find a way to make the 16-lane design's wave propagate slowly enough to be demonstrable. We would also consider adding another output such as the energy density  $u$ , as well as potentially swapping UPML for CPML. We would also look to integrate the conducting paper and probe to measure and visualise the electrostatic potential, which would require grounded boundaries instead of the absorbing PML. Switching to a more capable board with more BRAMs and logic resources would allow us to use a wider number format to extend the dynamic range, and clocking the solver faster would raise the simulation rate. We may also add a permittivity map to allow dielectric, conductor, and material

interfaces to be simulated. For the renderer, we would work on adding a variable step size such that steps become larger for distant terrain and smaller near the camera, further optimising the design without sacrificing accuracy.